



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2025–2026

Section: R

Project Report On

PharmaLink – An online pharmacy marketplace that connects patients with licensed neighbourhood pharmacies.

Supervised By

Dr. Md. Iftekharul Mobin

Submitted By:

Name	ID
1. Nafiul Islam	21-45717-3
2. Md Arafat Rahman	22-47910-2
3. Muhtasim Mahin	23-53789-3
4. Shohidur Raza Sujon	22-49449-3

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1–2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				Total =
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.				
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.				
Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.				

CO3: Prepare and explain a real-life desktop application by synthesizing C# components and development tools with database integration.					
Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1–2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				Total =
Organization of the application	Assessing whether the application is organized properly, with a complete report, a clear repository structure and an honest record of each member's contribution.				
Representation and Integration of Database	Assessing the representation of the database design, its normalization up to third normal form, and the integration of the tables through primary and foreign keys.				
Graphical User Interface	Assessing whether the graphical user interface is consistent, user friendly and coherent, and whether every form is reachable and has a way back.				

Date of Submission: _____

Table Of Contents

Topics	Page No.
1. Title	4
2. Introduction	4
3. Case Study	4
Functional Requirements & User Stories	5
4. ER Diagram	14
5. Normalization & Finalization	14
6. Database Schema	22
7. Database Query	27
8. Transition Table	45
9. Features and Facilities	58
10. Conclusion	60

1. Title

PharmaLink – An online pharmacy marketplace that connects patients with licensed neighbourhood pharmacies.

2. Introduction

In a city where a fever at eleven at night still means a rickshaw ride to whichever pharmacy is open, getting the right medicine quickly is a real problem and not a small one. PharmaLink is a desktop application, written in C# with Windows Forms over a Microsoft SQL Server database, that puts a software platform between the two sides of that problem. Licensed pharmacies list what is actually on their shelves, patients search across every listed pharmacy at once, and the company that runs the platform earns a commission on each completed sale.

The system is a three tier marketplace. The Super Admin is the platform operator, the Admin is a pharmacy owner selling through the platform, and the Customer is the patient buying. All three are stored in one Users table, and a UserType column decides which dashboard opens after login. This report covers the design phase: the case study the design was drawn from, the functional requirements and user stories for all three roles, the entity relationship diagram, the normalization of every relationship up to third normal form, the resulting database schema and the script that creates it, the transitions between every form in the application, and the features the finished system will provide.

3. Case Study

Anyone who has bought medicine at short notice in Dhaka knows the routine. A child spikes a fever at eleven at night, and someone goes out on a rickshaw to Mitford or the nearest para shop to find out whether the medicine is even in stock. If it is not, the search starts again at the next shop, with no way to compare prices and no way to confirm the strip is genuine. The pharmacy owner has the mirror problem. He keeps stock in a paper register, does not know which item is running out until a customer asks, and cannot reach a buyer four streets away who has never heard of his shop.

PharmaLink is a Windows desktop application that puts a software platform between those two parties. It owns no medicine; it is the IT company in the middle, as Chaldal sits between grocers and households. Licensed pharmacies list what is on their shelves, patients search every listed pharmacy at once, and the platform takes a commission on each sale.

Three actors work inside the system and each wants something different. The Super Admin is the platform operator. He wants a trustworthy marketplace, so he approves every new pharmacy against its DGDA drug licence number, suspends the ones customers rate badly, maintains the master category list, moderates abusive reviews and watches the commission earned. The Admin is a pharmacy owner such as the proprietor of Mitford Pharma. He wants to sell more without hiring staff, so he lists his medicines with price and stock, is warned when an item falls below its

minimum level, runs discount offers, and reads an earnings report telling him what he is owed. The Customer is a patient such as a resident of Mirpur 10. She wants the right medicine at a fair price delivered to her door, so she searches, filters by category, price range and area, reads what other buyers said, pays by bKash or cash on delivery, and rates the pharmacy afterwards.

The entities are few and clear. A user is identified by a user id and carries a full name, a unique email, a hashed password, a phone number, a user type that decides which dashboard opens after login, and an account status. A pharmacy is identified by a pharmacy id and holds a shop name, a unique drug licence number, an area such as Mitford or Dhanmondi, a commission rate and an approval status. A medicine is identified by a medicine id and holds a brand name, a generic name, a manufacturer, a strength, a unit price, current stock, a minimum stock level, a prescription required flag and an expiry date.

The relationships are what make the system worth building. One user owns exactly one pharmacy, one pharmacy lists many medicines, and one category classifies many medicines. One customer places many orders, and one order contains many medicines while one medicine appears in many orders, so that many to many relationship is resolved through the OrderItems junction table. One customer writes many reviews, and each review points back to the order that proves the purchase was real.

Money flows in one direction and the commission comes out of the middle of it. The customer pays the full bill at checkout. The order stores the total amount and, beside it, the commission calculated from that pharmacy's own rate, eight percent for most shops and ten for a few. PharmaLink settles the remainder to the owner. Because the commission is frozen on the order row, raising prices next month cannot change what a pharmacy owes on last month's sales.

Functional Requirements & User Stories

The requirements are numbered and grouped by role. Every one of them is answered by a form in the transition table in Section 8 and by at least one statement in the script in Section 7.

Super Admin

No.	Requirement
1	The Super Admin shall log in with the same login form as every other user and be routed to the Super Admin dashboard when UserType = 'SuperAdmin'.
2	The Super Admin shall view every pharmacy registration that is in Pending status and approve or reject it after checking the DGDA licence number.
3	The Super Admin shall suspend or delete a pharmacy owner, especially one whose average customer rating is poor.
4	The Super Admin shall view a list of all Admins and Customers with a keyword search and a status filter.
5	The Super Admin shall view a platform wide sales dashboard showing total orders, total revenue and commission earned per pharmacy.
6	The Super Admin shall generate a low rated pharmacy report listing every pharmacy whose average rating is below 2.5.
7	The Super Admin shall add, edit and deactivate entries in the master medicine category list.

8	The Super Admin shall hide abusive or fake reviews without deleting the underlying row.
9	The Super Admin shall set the commission rate of an individual pharmacy between 0 and 30 percent.

Admin (Pharmacy Owner)

No.	Requirement
10	The Admin shall register a pharmacy with a shop name, DGDA licence number, area, address, contact number and logo, and the pharmacy shall stay Pending until the Super Admin approves it.
11	The Admin shall create, view, update and delete medicines belonging only to his own pharmacy, through a DataGridView.
12	The Admin shall view a stock dashboard showing units sold, units remaining and a low stock alert for every medicine where Stock is less than MinStock.
13	The Admin shall generate a sales and earnings report showing who bought what, on which date and time, at what unit price, with gross sales, platform commission and net earnings.
14	The Admin shall create percentage discount offers on his own medicines with a start date and an end date.
15	The Admin shall read the ratings and reviews written on his own medicines but shall not be able to edit or delete them.
16	The Admin shall verify or reject a prescription image uploaded against an order that contains a prescription only medicine.
17	The Admin shall update his own profile information and change his own password.
18	The system shall filter every Admin side query by the logged in owner's PharmacyId so that no pharmacy can read another pharmacy's data.

Customer

No.	Requirement
19	The Customer shall sign up with a full name, unique email, unique mobile number, address and password, and shall sign in through the shared login form.
20	The Customer shall browse medicines listed by every approved pharmacy on one screen.
21	The Customer shall search medicines by brand name, generic name or manufacturer.
22	The Customer shall narrow the result set using at least three ComboBox filters: category, price range, area, pharmacy and availability.
23	The Customer shall open a medicine details screen showing the manufacturer, strength, expiry date, selling pharmacy, prescription requirement and all visible reviews.
24	The Customer shall add a medicine to the cart, change the quantity of a cart line and remove a cart line.
25	The Customer shall check out by entering a delivery address and choosing bKash, Nagad, card or cash on delivery, which generates a printable invoice.
26	The Customer shall upload a photograph of a doctor's prescription when the cart contains a medicine whose RequiresRx flag is set.
27	The Customer shall view a history of past orders with the invoice of each one.
28	The Customer shall give a rating from 1 to 5 with a written comment, once per medicine per delivered order.
29	The Customer shall view all discount offers that are active today with the discounted price already calculated.
30	The Customer shall update her own profile and change her own password.

Traceability

No requirement is left without a screen, and no screen is left without a query.

No.	Form	Query	No.	Form	Query
1	LoginForm	7.1	16	VerifyPrescriptionForm	7.18
2	SuperAdminManageShopsForm	7.11	17	AdminProfileForm	7.18
3	SuperAdminManageShopsForm	7.11	18	every Admin form	7.7, 7.8, 7.15
4	SuperAdminManageUsersForm	7.16	19	SignUpForm, LoginForm	7.14, 7.1
5	SuperAdminSalesReportForm	7.7, 7.13	20	CustomerHomeForm	7.2, 7.3
6	SuperAdminLowRatedShopsForm	7.10	21	shared search box	7.4
7	ManageCategoriesForm	7.16	22	CustomerHomeForm	7.2, 7.3
8	ModerateReviewsForm	7.16	23	MedicineDetailsForm	7.9
9	SuperAdminManageShopsForm	7.16	24	CartForm	7.5
10	SignUpForm, PharmacyProfileForm	7.14	25	CheckoutForm, InvoiceForm	7.6
11	AdminMedicineForm, Add / Edit Medicine	7.15	26	UploadPrescriptionForm	7.18
12	AdminInventoryForm	7.8	27	OrderHistoryForm	7.17
13	AdminEarningsForm	7.7	28	GiveRatingForm	7.17
14	DiscountOffersForm	7.15	29	CustomerOffersForm	7.12
15	AdminReviewsForm	7.9	30	MyProfileForm	7.18

User Stories

Each story is written in the form *As a role, I can do something, so that business benefit*, followed by the detail of the form, its inputs and buttons, what is validated and what happens in the database.

Super Admin

1. As a Super Admin, I can sign in through the same login form as everyone else, so that the platform has one entry point and one place where access is decided.

There is no separate administrator login. The Super Admin types the same email and password into LoginForm, and the login query returns his UserType along with his account status. The form reads that value and opens the Super Admin dashboard; the same statement would open a pharmacy dashboard or a customer home for a different UserType. An account whose Status is Pending or Suspended is refused by the same WHERE clause that checks the password, so a suspended administrator cannot get in even with the correct credentials.

2. As a Super Admin, I can search the full user list, so that I can find an account when a customer or an owner contacts support.

Manage Users shows every Admin and Customer in one DataGridView, with the pharmacy name filled in beside an owner's row through a LEFT JOIN on Pharmacies. A free text box searches name and email, and a Status ComboBox filters to Pending, Active or Suspended; both are optional, and an empty box means no filter rather than no results. Selecting a row and clicking Suspend runs the same status update used on the Manage Pharmacies form, so there is one rule for suspension in the whole application.

3. As a Super Admin, I can change one pharmacy's commission rate, so that I can offer a better rate to a high volume shop without touching anyone else.

The rate is a column on Pharmacies rather than a constant in the code, and it is edited from the Manage Pharmacies form. The value must be between 0 and 30, which is enforced by the form and again by a CHECK constraint. Changing it affects only orders placed from that moment on, because every order stores its own CommissionAmount at checkout time, so last month's settlement is never rewritten by today's decision.

4. As a Super Admin, I can approve or reject a new pharmacy registration, so that only pharmacies with a valid drug licence can sell on the platform.

The Super Admin opens Manage Pharmacies from the left menu. A DataGridView lists every pharmacy with its owner, licence number, area and status, and a Status ComboBox filters the grid down to Pending. Selecting a row enables the Approve and Reject buttons; with no row selected both stay disabled. Approve runs an UPDATE that sets Pharmacies.Status to 'Approved' and Users.Status to 'Active', after which the owner can log in and the pharmacy's medicines become visible to customers.

5. As a Super Admin, I can suspend a pharmacy owner, so that a shop with repeated complaints stops receiving new orders.

The Super Admin selects a pharmacy row and clicks Suspend. A modal confirmation dialog opens showing the pharmacy name and its average rating, because suspension is not reversible from the customer's point of view within the same session. On Yes, three UPDATE statements run inside one transaction: Pharmacies.Status becomes 'Suspended', Users.Status becomes 'Suspended', and every row in Medicines belonging to that pharmacy has IsActive set to 0. Past orders are never deleted, so the sales history and the invoices customers already hold stay valid.

6. As a Super Admin, I can see which pharmacies are rated below 2.5, so that I can act on poor service before customers leave the platform.

The Low-Rated Pharmacies form runs a single query that joins Pharmacies, Medicines and Reviews, groups by pharmacy and applies `HAVING AVG(Rating) < 2.5 AND COUNT(ReviewId) >= 2`, so that one angry review cannot condemn a shop. The grid shows the pharmacy, owner, area, review count and average rating, with rows tinted red. Double clicking a row opens that pharmacy in the Manage Pharmacies form with the record already selected.

7. As a Super Admin, I can view total revenue and commission per pharmacy, so that I know what the platform has earned and what it owes.

The Platform Sales Report form takes a date range, an area and an order status through ComboBox and DateTimePicker controls. Generate runs a query that joins Pharmacies, Orders and OrderItems, groups by pharmacy and returns order count, units sold, gross sales, commission and average item price, with a bold total row appended in the DataGridView. Export CSV writes the same result set to a file so it can be reconciled against bank settlements.

8. As a Super Admin, I can hide an abusive review, so that the review section stays useful without destroying the audit trail.

The Moderate Reviews form defaults its Rating filter to one and two star reviews, which is where abuse usually sits. The grid shows the reviewer, medicine, pharmacy, comment, date and the order id that proves the purchase. Hide Review sets Reviews.IsHidden to 1 rather than deleting the row, so the review disappears from the customer screens and from the average rating calculation, but remains available if the pharmacy disputes the decision.

9. As a Super Admin, I can maintain the master category list, so that every pharmacy classifies its medicines the same way.

The Manage Categories form is a simple DataGridView with Add, Edit and Deactivate buttons. Category name is validated as non empty and is protected by a UNIQUE constraint, so a second 'Antibiotic' row is rejected with a red error label instead of a database exception. A category that is already referenced by a medicine cannot be deleted; it can only be deactivated by setting IsActive to 0, which keeps existing foreign keys valid.

Admin (Pharmacy Owner)

10. As a Pharmacy Owner, I can register my pharmacy and wait for approval, so that customers can trust that every shop on the platform holds a real drug licence.

The Sign Up form, with Register as set to Pharmacy Owner, asks for the shop name, DGDA licence number, area, address and contact number in addition to the personal fields. On save two rows are written inside one transaction, a Users row with UserType 'Admin' and Status 'Pending' and a Pharmacies row with Status 'Pending' linked to it. The licence number is protected by a UNIQUE constraint, so the same licence cannot be registered twice. Until the Super Admin approves the application the owner cannot log in and none of his medicines are visible to customers.

11. As a Pharmacy Owner, I can update my shop profile and my password, so that customers always see my current address and my account stays secure.

The My Pharmacy Profile form edits the shop name, area, address, contact number and logo, and every UPDATE carries WHERE PharmacyId = @PharmacyId so an owner cannot edit another shop. The licence number is displayed read only, because changing it would mean a new licence and a fresh approval. The Change Password panel asks for the current password, a new one of at least six characters containing a digit, and a confirmation; the update succeeds only if the stored hash matches the current password, so a wrong entry simply updates no rows and the form reports failure.

12. As a Pharmacy Owner, I can add a new medicine, so that customers can find and buy it from my pharmacy.

The owner clicks Add Medicine on the Medicines form and a modal dialog opens asking for medicine name, generic name, category (ComboBox), manufacturer, strength, unit price, stock, minimum stock, expiry date, a prescription required checkbox and a description. Unit price must be a positive number, stock and minimum stock must be zero or more, expiry date must be in the future and the name cannot be empty; if any rule fails a red error label appears under the field and the Save button stays disabled. On Save a row is inserted into Medicines carrying the logged in owner's PharmacyId, the same rules are enforced again by CHECK constraints in the database, and the DataGridView refreshes immediately.

13. As a Pharmacy Owner, I can see which of my medicines are running out, so that I can restock before I lose a sale.

The Stock and Inventory form opens with four summary tiles and a Low Stock Alert panel. The alert grid runs a query filtered by WHERE PharmacyId = @PharmacyId AND Stock < MinStock, and it displays the shortfall in units so the owner knows how much to order. Rows in the alert panel are painted with a red background through the DataGridView CellFormatting event. Selecting a row and clicking Restock opens the Edit Medicine dialog with the stock field focused.

14. As a Pharmacy Owner, I can read my earnings report, so that I can check what PharmaLink owes me after commission.

The Sales and Earnings form takes a date range and an optional medicine filter. It joins Orders, OrderItems, Medicines and Users so that every line shows the order number, date and time, customer name, medicine, quantity, unit price and subtotal. Four tiles above the grid show gross sales, platform commission, net earnings and units sold. Commission is read from Orders.CommissionAmount, which was frozen at checkout time, so changing a medicine's price today never rewrites last month's report.

15. As a Pharmacy Owner, I can create a discount offer, so that I can move stock during a slow season.

The Discount Offers form lists the owner's existing offers and lets him create a new one by choosing a medicine from a ComboBox, entering a discount percentage and picking a start and end date. The percentage must be greater than 0 and no more than 70, and the end date cannot be earlier than the start date; both rules are enforced in the form and again by CHECK constraints on the Offers table. Once saved, the discounted price appears automatically on the customer's Offers screen for exactly the dates chosen.

16. As a Pharmacy Owner, I can read the reviews written about my medicines, so that I can understand what customers complain about.

The Customer Reviews form is read only by design. It shows a DataGridView of reviewer name, medicine, rating, comment and date for medicines belonging to this pharmacy only, with an average rating displayed above the grid. There is no Delete button anywhere on this form. If the owner believes a review is abusive he uses the Report button, which flags the review for the Super Admin rather than removing it himself.

17. As a Pharmacy Owner, I can verify a prescription before dispatch, so that I do not dispense a controlled medicine without a doctor's order.

When an order contains a medicine whose RequiresRx flag is set, the order appears in the Prescriptions queue with the uploaded image and the doctor's name. The owner opens the image, then clicks Approve or Reject, which sets Prescriptions.VerifyStatus. An order whose prescription is still Pending cannot be moved to Confirmed, so the Confirm button on that order stays disabled and a hint explains why.

Customer

18. As a Customer, I can look back at my past orders, so that I can reorder the same medicine and show a bill if something is wrong.

Order History lists every order with its date, the selling pharmacy, the number of line items, the total paid, the payment method and a status pill, filtered by status, pharmacy and date range through three ComboBox controls. View Invoice reopens the printable bill for the selected order. Because an order that spans two pharmacies was split at checkout, each pharmacy's delivery appears as its own row with its own invoice, which is what the customer actually received.

19. As a Customer, I can create an account, so that I can order medicine without visiting a pharmacy in person.

The Sign Up form asks for account type, full name, email, mobile number, address, password and password confirmation. Email must match a basic address pattern and must not already exist, mobile must be eleven digits and unique, and the two password boxes must match; each failure shows a red label directly under the offending field and keeps the Create Account button disabled. On success a row is inserted into Users with UserType 'Customer' and Status 'Active', while a pharmacy owner signing up is inserted with Status 'Pending' and must wait for Super Admin approval.

20. As a Customer, I can search and filter medicines, so that I can find what I need at a price I can afford.

The Home screen carries a search TextBox and five ComboBox filters: category, price range, area, pharmacy and availability. Search matches the keyword against medicine name, generic name and manufacturer with a LIKE query, so typing 'paracetamol' finds Napa and Ace Plus even though neither brand contains that word. Filters are combined in a single query and only medicines belonging to Approved pharmacies are returned. The result count and the number of active filters are shown in the status strip.

21. As a Customer, I can read reviews before I buy, so that I can avoid a pharmacy that sends damaged goods.

The Medicine Details screen shows the manufacturer, strength, expiry date, selling pharmacy and whether a prescription is needed, followed by a grid of every visible review with the reviewer's name, star rating, comment and date. The reviews come from a join of Reviews and Users filtered by IsHidden = 0. If a discount offer is running today the original price is struck through and the discounted price is displayed beside it.

22. As a Customer, I can manage my cart, so that I can change my mind before paying.

The Cart form lists each line with the medicine, selling pharmacy, unit price, a quantity spinner and a subtotal, plus a summary panel showing items total, discount, delivery charge and the payable amount. Increasing a quantity above the available stock is refused with a message, and reducing quantity to zero removes the line. Adding a medicine that is already in the cart updates the existing row instead of creating a duplicate, which the UNIQUE constraint on (CustomerId, MedicineId) guarantees.

23. As a Customer, I can check out and receive an invoice, so that I have proof of what I paid.

The Checkout form pre fills the delivery address from the profile and asks the customer to pick a payment method; choosing bKash or Nagad reveals a mobile number field that must be eleven digits, and the Confirm Order button stays disabled until every field is valid. Confirm runs one transaction that inserts the Orders row with its total and commission, inserts one OrderItems row per cart line at the price shown on screen, decrements Medicines.Stock, and deletes the customer's cart. The Invoice screen then opens with a printable bill carrying the order number, both addresses, the line items and the grand total.

24. As a Customer, I can rate a medicine after delivery, so that other patients know whether the pharmacy is reliable.

The Rate and Review button on the Order History form is enabled only for orders whose status is 'Delivered' and that have not been reviewed yet. It opens a modal with a one to five star selector and a comment box limited to 500 characters; a rating must be chosen before Submit is enabled. Submit inserts a row into Reviews carrying the customer id, medicine id and order id, and the UNIQUE constraint on those three columns stops the same purchase being rated twice.

25. As a Customer, I can upload my prescription, so that I can buy an antibiotic that legally needs one.

If the cart contains a medicine with RequiresRx set, the Confirm Order step opens a modal asking for a photograph of the prescription and, optionally, the prescribing doctor's name. Only JPG and PNG files under 2 MB are accepted and the dialog cannot be dismissed without either uploading or removing the medicine from the cart. The file path is stored in Prescriptions with VerifyStatus 'Pending', and the order waits in the pharmacy's verification queue before it is dispatched.

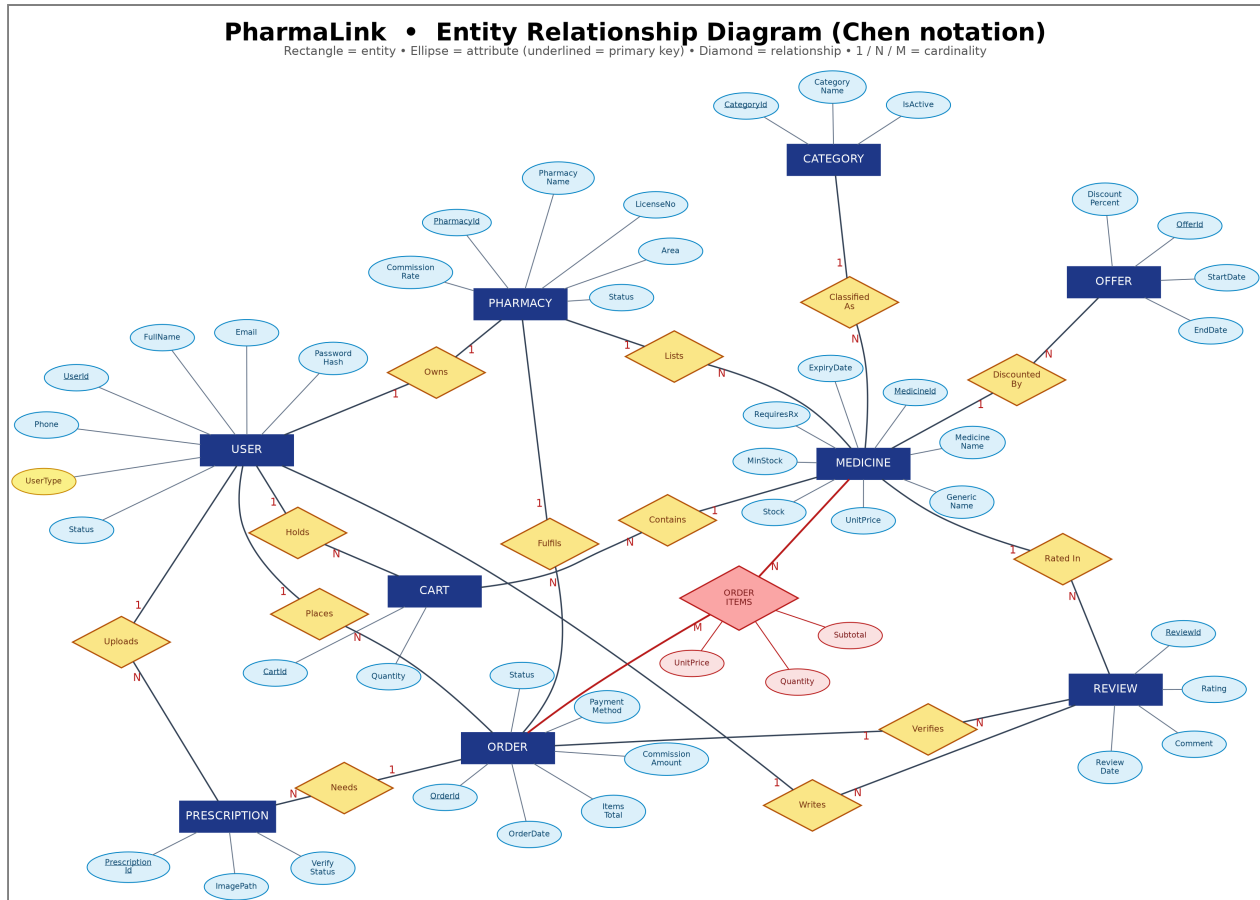
26. As a Customer, I can see today's offers, so that I can buy my regular medicine when it is cheapest.

The Offers screen runs a query that returns only offers where today falls between StartDate and EndDate, the medicine is in stock and the pharmacy is Approved. The grid shows the offer title, medicine, pharmacy, original price, discount percentage, the calculated price the customer will pay and the last valid date. Expired offers are filtered out by the query rather than by the form, so nothing stale can ever be displayed.

27. As a Customer, I can change my password, so that my account stays secure.

The My Profile form has a Change Password panel asking for the current password, a new password and a confirmation. The new password must be at least six characters and contain at least one digit, and the two new entries must match. The current password is verified against the stored salted SHA-256 hash before anything is written, and on success only Users.PasswordHash and Users.PasswordSalt are updated. Plain text passwords are never stored or logged.

4. ER Diagram



Chen notation. Rectangles are entities, ellipses are attributes with the primary key underlined, diamonds are relationships, and 1, N and M give the cardinality.

5. Normalization & Finalization

Every relationship on the ER diagram in Section 4 is normalised below. The unnormalised form (UNF) of a relationship is the joined attribute set of the two entities it connects, so an attribute that exists on both sides is written with a qualifier here (for example UserAddress and PharmacyAddress) purely to keep the two apart while they sit in the same relation. Once the relation is decomposed each of them is simply Address in its own table. Underlined attributes are primary keys and attributes marked (FK) are foreign keys.

Owns: one to one relationship.

UNF: UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt, PharmacyId, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt.

1NF: UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt, PharmacyId, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt.

2NF:

1. **UserId**, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt.
2. **PharmacyId**, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt, **UserId**_(FK).

3NF: Same as 2NF.

OwnerId in the final table is this UserId foreign key. It is also declared UNIQUE, which is what turns a one to many foreign key into the one to one relationship the ER diagram shows: a pharmacy owner owns exactly one pharmacy.

Lists: one to many relationship.

UNF: PharmacyId, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt, **MedicineId**, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, CategoryId, CategoryName.

1NF: PharmacyId, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt, **MedicineId**, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, CategoryId, CategoryName.

2NF:

1. **PharmacyId**, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt.
2. **MedicineId**, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, CategoryId, CategoryName, **PharmacyId**_(FK).

3NF:

1. **PharmacyId**, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt.
2. **MedicineId**, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, **CategoryId**_(FK), **PharmacyId**_(FK).
3. **CategoryId**, CategoryName, CategoryDescription, IsActive.

This is the one relationship where 3NF does real work. In the 2NF form, MedicineId determines CategoryId and CategoryId in turn determines CategoryName, so CategoryName depends on the key only through another non key attribute. That is a transitive dependency, and it is removed by moving the category name into its own relation. Without this step, renaming a category would mean an UPDATE on every medicine row that carries it.

Classified As: one to many relationship.

UNF: CategoryId, CategoryName, CategoryDescription, IsActive, MedicineId, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive.

1NF: CategoryId, CategoryName, CategoryDescription, IsActive, MedicineId, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive.

2NF:

1. CategoryId, CategoryName, CategoryDescription, IsActive.
2. MedicineId, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, CategoryId_(FK).

3NF: Same as 2NF.

Holds: one to many relationship.

UNF: UserId, FullName, Email, Phone, UserType, Status, CartId, Quantity, AddedDate, MedicineId, MedicineName, UnitPrice.

1NF: UserId, FullName, Email, Phone, UserType, Status, CartId, Quantity, AddedDate, MedicineId, MedicineName, UnitPrice.

2NF:

1. UserId, FullName, Email, Phone, UserType, Status.
2. CartId, Quantity, AddedDate, MedicineId, MedicineName, UnitPrice, UserId_(FK).

3NF:

1. UserId, FullName, Email, Phone, UserType, Status.
2. CartId, Quantity, AddedDate, MedicineId_(FK), UserId_(FK).
3. MedicineId, MedicineName, GenericName, UnitPrice, Stock.

MedicineName and UnitPrice depend on MedicineId rather than on CartId, so they are transitively dependent on the key and are moved back to the medicine relation. The cart line then stores only what is true of the line itself: which customer, which medicine, how many, and when it was added.

Contains: one to many relationship.

UNF: MedicineId, MedicineName, GenericName, UnitPrice, Stock, CartId, Quantity, AddedDate.

1NF: MedicineId, MedicineName, GenericName, UnitPrice, Stock, CartId, Quantity, AddedDate.

2NF:

1. MedicineId, MedicineName, GenericName, UnitPrice, Stock.
2. CartId, Quantity, AddedDate, MedicineId_(FK).

3NF: Same as 2NF.

Places: one to many relationship.

UNF: UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt, OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus.

1NF: UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt, OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus.

2NF:

1. UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt.
2. OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus, UserId_(FK).

3NF: Same as 2NF.

TotalAmount does not appear in this list because it is not stored as an independent attribute. It is declared as a computed PERSISTED column equal to ItemsTotal + DeliveryCharge, so the bill total is derived by the database engine and can never disagree with its two parts.

Fulfil: one to many relationship.

UNF: PharmacyId, PharmacyName, LicenseNo, Area, CommissionRate, PharmacyStatus, OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus.

1NF: PharmacyId, PharmacyName, LicenseNo, Area, CommissionRate, PharmacyStatus, OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus.

2NF:

1. PharmacyId, PharmacyName, LicenseNo, Area, CommissionRate, PharmacyStatus.
2. OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus, PharmacyId_(FK).

3NF: Same as 2NF.

A basket that holds medicines from two pharmacies produces two orders, one per pharmacy, which is why PharmacyId sits on the order header rather than on the individual line items.

Order Items: many to many relationship.

UNF: OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus, { MedicineId, MedicineName, UnitPrice, Quantity, Subtotal }.

1NF: OrderId, MedicineId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus, MedicineName, UnitPrice, Quantity, Subtotal.

2NF:

1. OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus.

2. **MedicineId**, MedicineName, GenericName, Manufacturer, UnitPrice, Stock.

3. **OrderId**, **MedicineId**, Quantity, UnitPrice, Subtotal.

3NF: Same as 2NF.

This is the only many to many relationship in the model and it is the one the assignment brief calls out. One order contains many medicines and one medicine appears in many orders, so in UNF the medicine details form a repeating group inside the order, shown here in braces. 1NF removes the repeating group by giving every medicine of an order its own row, which makes the key the composite (OrderId, MedicineId). That composite key then creates partial dependencies: OrderDate and PaymentMethod depend on OrderId alone, while MedicineName depends on MedicineId alone. 2NF removes both by splitting the relation into three, and the third of them is the OrderItems junction table. Quantity, UnitPrice and Subtotal stay on the junction because they depend on the whole composite key: they are the quantity of that medicine in that order at the price charged on that day. Subtotal is stored as a computed PERSISTED column equal to Quantity times UnitPrice, so it is derived by the engine rather than kept independently.

Writes: one to many relationship.

UNF: **UserId**, FullName, Email, Phone, UserType, Status, **ReviewId**, Rating, Comment, ReviewDate, IsHidden.

1NF: **UserId**, FullName, Email, Phone, UserType, Status, **ReviewId**, Rating, Comment, ReviewDate, IsHidden.

2NF:

1. **UserId**, FullName, Email, Phone, UserType, Status.

2. **ReviewId**, Rating, Comment, ReviewDate, IsHidden, **UserId**_(FK).

3NF: Same as 2NF.

Rated In: one to many relationship.

UNF: **MedicineId**, MedicineName, GenericName, UnitPrice, Stock, **ReviewId**, Rating, Comment, ReviewDate, IsHidden.

1NF: **MedicineId**, MedicineName, GenericName, UnitPrice, Stock, **ReviewId**, Rating, Comment, ReviewDate, IsHidden.

2NF:

1. **MedicineId**, MedicineName, GenericName, UnitPrice, Stock.

2. **ReviewId**, Rating, Comment, ReviewDate, IsHidden, **MedicineId**_(FK).

3NF: Same as 2NF.

Verifies: one to many relationship.

UNF: **OrderId**, OrderDate, ItemsTotal, OrderStatus, **ReviewId**, Rating, Comment, ReviewDate.

1NF: **OrderId**, OrderDate, ItemsTotal, OrderStatus, **ReviewId**, Rating, Comment, ReviewDate.

2NF:

1. **OrderId**, OrderDate, ItemsTotal, OrderStatus.

2. **ReviewId**, Rating, Comment, ReviewDate, **OrderId**_(FK).

3NF: Same as 2NF.

The review carries the order it came from as well as the customer and the medicine. That third foreign key is what makes a review verifiable: the application can prove the reviewer actually bought the item, and the UNIQUE constraint on the three columns together stops the same purchase being rated twice.

Discounted By: one to many relationship.

UNF: MedicineId, MedicineName, GenericName, UnitPrice, Stock, OfferId, OfferTitle, DiscountPercent, StartDate, EndDate, OfferIsActive.

1NF: MedicineId, MedicineName, GenericName, UnitPrice, Stock, OfferId, OfferTitle, DiscountPercent, StartDate, EndDate, OfferIsActive.

2NF:

1. MedicineId, MedicineName, GenericName, UnitPrice, Stock.
2. OfferId, OfferTitle, DiscountPercent, StartDate, EndDate, OfferIsActive, MedicineId_(FK).

3NF: Same as 2NF.

Needs: one to many relationship.

UNF: OrderId, OrderDate, ItemsTotal, OrderStatus, PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus.

1NF: OrderId, OrderDate, ItemsTotal, OrderStatus, PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus.

2NF:

1. OrderId, OrderDate, ItemsTotal, OrderStatus.
2. PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus, OrderId_(FK).

3NF: Same as 2NF.

Uploads: one to many relationship.

UNF: UserId, FullName, Phone, PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus.

1NF: UserId, FullName, Phone, PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus.

2NF:

1. UserId, FullName, Phone.
2. PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus, UserId_(FK).

3NF: Same as 2NF.

Finalization

1. **Users:** UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt.

2. **Pharmacies:** PharmacyId, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt, UserId_(FK).
3. **Pharmacies:** PharmacyId, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt.
4. **Medicines:** MedicineId, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, CategoryId_(FK), PharmacyId_(FK).
5. **Categories:** CategoryId, CategoryName, CategoryDescription, IsActive.
6. **Categories:** CategoryId, CategoryName, CategoryDescription, IsActive.
7. **Medicines:** MedicineId, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, CategoryId_(FK).
8. **Users:** UserId, FullName, Email, Phone, UserType, Status.
9. **Cart:** CartId, Quantity, AddedDate, MedicineId_(FK), UserId_(FK).
10. **Medicines:** MedicineId, MedicineName, GenericName, UnitPrice, Stock.
11. **Medicines:** MedicineId, MedicineName, GenericName, UnitPrice, Stock.
12. **Cart:** CartId, Quantity, AddedDate, MedicineId_(FK).
13. **Users:** UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt.
14. **Orders:** OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus, UserId_(FK), PharmacyId_(FK).
15. **Pharmacies:** PharmacyId, PharmacyName, LicenseNo, Area, CommissionRate, PharmacyStatus.
16. **Orders:** OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus, PharmacyId_(FK).
17. **Orders:** OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus.
18. **Medicines:** MedicineId, MedicineName, GenericName, Manufacturer, UnitPrice, Stock.
19. **OrderItems:** OrderId, MedicineId, Quantity, UnitPrice, Subtotal.
20. **Users:** UserId, FullName, Email, Phone, UserType, Status.
21. **Reviews:** ReviewId, Rating, Comment, ReviewDate, IsHidden, UserId_(FK), MedicineId_(FK), OrderId_(FK).
22. **Medicines:** MedicineId, MedicineName, GenericName, UnitPrice, Stock.
23. **Reviews:** ReviewId, Rating, Comment, ReviewDate, IsHidden, MedicineId_(FK).
24. **Orders:** OrderId, OrderDate, ItemsTotal, OrderStatus.
25. **Reviews:** ReviewId, Rating, Comment, ReviewDate, OrderId_(FK).
26. **Medicines:** MedicineId, MedicineName, GenericName, UnitPrice, Stock.

27. **Offers:** OfferId, OfferTitle, DiscountPercent, StartDate, EndDate, OfferIsActive, MedicineId_(FK).
28. **Orders:** OrderId, OrderDate, ItemsTotal, OrderStatus.
29. **Prescriptions:** PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus, OrderId_(FK), UserId_(FK).
30. **Users:** UserId, FullName, Phone.
31. **Prescriptions:** PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus, UserId_(FK).

The list above collects every relation produced by the fourteen decompositions, in the order they were produced. Entries shown in red are repeats: the same entity appears in several relationships, so its relation is derived more than once. Where two versions of the same entity differ, the fuller one is kept and the foreign keys of the shorter one are merged into it, which is why Orders carries both UserId and PharmacyId, Reviews carries all three of its foreign keys, and Prescriptions carries two. Removing the repeats leaves the ten tables below.

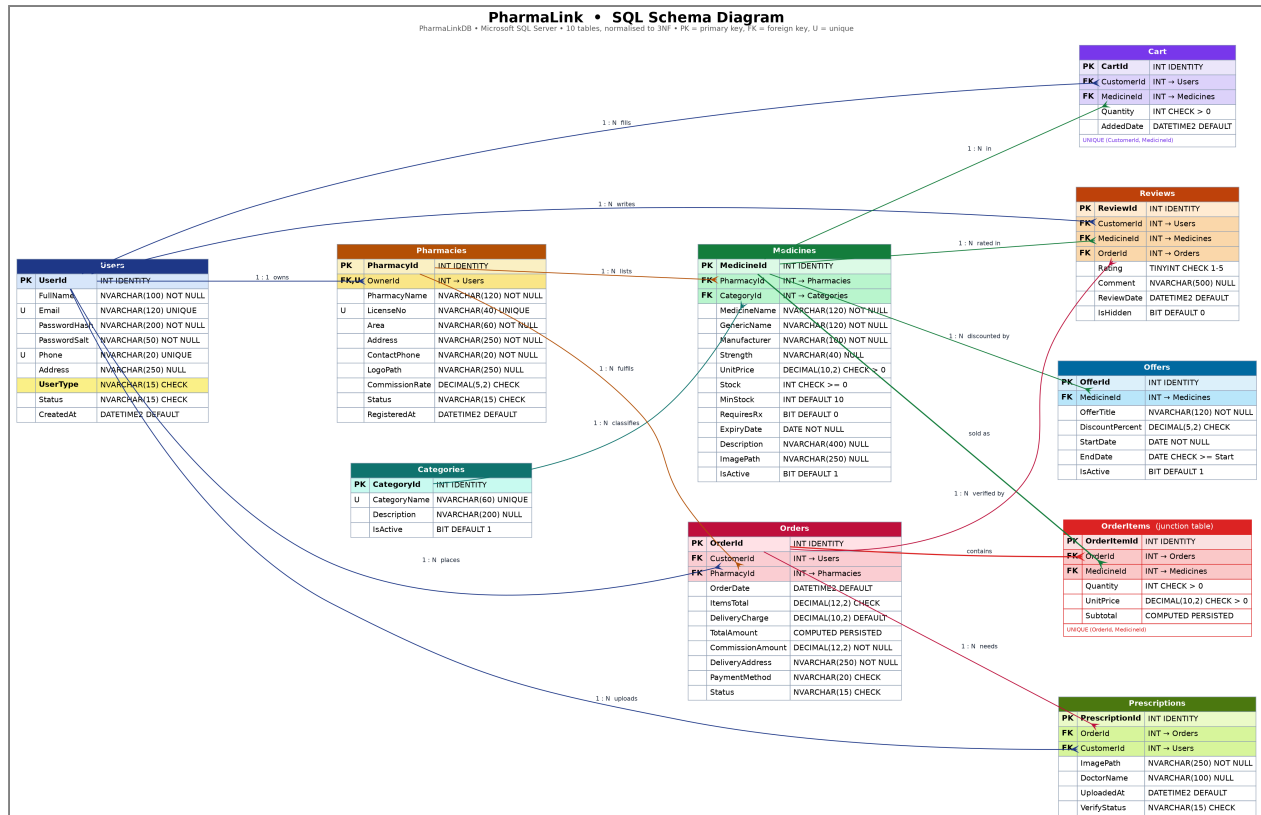
Final Tables

1. **Users:** UserId, FullName, Email, PasswordHash, PasswordSalt, Phone, UserAddress, UserType, Status, CreatedAt.
2. **Categories:** CategoryId, CategoryName, CategoryDescription, IsActive.
3. **Pharmacies:** PharmacyId, PharmacyName, LicenseNo, Area, PharmacyAddress, ContactPhone, LogoPath, CommissionRate, PharmacyStatus, RegisteredAt, UserId_(FK).
4. **Medicines:** MedicineId, MedicineName, GenericName, Manufacturer, Strength, UnitPrice, Stock, MinStock, RequiresRx, ExpiryDate, MedicineDescription, ImagePath, MedicineIsActive, PharmacyId_(FK), CategoryId_(FK).
5. **Cart:** CartId, Quantity, AddedDate, UserId_(FK), MedicineId_(FK).
6. **Orders:** OrderId, OrderDate, ItemsTotal, DeliveryCharge, CommissionAmount, DeliveryAddress, PaymentMethod, OrderStatus, UserId_(FK), PharmacyId_(FK).
7. **OrderItems:** OrderId, MedicineId, Quantity, UnitPrice, Subtotal.
8. **Reviews:** ReviewId, Rating, Comment, ReviewDate, IsHidden, UserId_(FK), MedicineId_(FK), OrderId_(FK).
9. **Offers:** OfferId, OfferTitle, DiscountPercent, StartDate, EndDate, OfferIsActive, MedicineId_(FK).
10. **Prescriptions:** PrescriptionId, ImagePath, DoctorName, UploadedAt, VerifyStatus, OrderId_(FK), UserId_(FK).

These ten relations are the tables created by the script in Section 7. Two changes are made when they reach SQL Server. First, the qualifiers used above to keep clashing names apart are dropped, so UserAddress and PharmacyAddress both become Address in their own table, and OrderStatus, PharmacyStatus, MedicineIsActive and OfferIsActive become Status and IsActive. Second, OrderItems is given a surrogate OrderItemId primary key for convenience in C#, and its natural

composite key (OrderId, MedicineId) is protected by a UNIQUE constraint instead, so the 2NF argument above still holds exactly.

6. Database Schema



PharmaLinkDB. PK marks a primary key, FK a foreign key and U a unique constraint. OrderItems, outlined in red, is the junction table produced by the Order Items decomposition in Section 5.

6.1 Table Descriptions

1. Users – Stores all three roles in one table. The UserType column is what the login query reads to decide which dashboard opens.

Column	Data Type	Constraint	Description
UserId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key for every person on the platform
FullName	NVARCHAR(100)	NOT NULL	Display name shown on dashboards, invoices and reviews
Email	NVARCHAR(120)	UNIQUE, NOT NULL, CHECK	Login identifier; CHECK enforces a basic address pattern
PasswordHash	NVARCHAR(200)	NOT NULL	Salted SHA-256 hash; plain text is never stored
PasswordSalt	NVARCHAR(50)	NOT NULL	Per user random salt used when hashing
Phone	NVARCHAR(20)	UNIQUE, NOT NULL	Contact number, unique so one number is one account
Address	NVARCHAR(250)	NULL	Default delivery address, pre filled at checkout

UserType	NVARCHAR(15)	NOT NULL, CHECK	'SuperAdmin', 'Admin' or 'Customer'; drives dashboard routing
Status	NVARCHAR(15)	NOT NULL, DEFAULT, CHECK	'Pending', 'Active' or 'Suspended'; only Active users can log in
CreatedAt	DATETIME2(0)	NOT NULL, DEFAULT	Registration timestamp, shown as Member Since

2. Categories – Master list of medicine categories maintained by the Super Admin. Kept separate so a category name is stored once.

Column	Data Type	Constraint	Description
CategoryId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key
CategoryName	NVARCHAR(60)	UNIQUE, NOT NULL	For example Antibiotic, Painkiller, Diabetes Care
Description	NVARCHAR(200)	NULL	Short explanation shown as a tooltip in the filter dropdown
IsActive	BIT	NOT NULL, DEFAULT 1	Soft delete flag; a referenced category is deactivated, never removed

3. Pharmacies – One row per Admin. OwnerId is UNIQUE, which enforces the rule that one pharmacy owner owns exactly one pharmacy.

Column	Data Type	Constraint	Description
PharmacyId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key; every Admin side query filters on this value
OwnerId	INT	FOREIGN KEY, UNIQUE	References Users(UserId); UNIQUE gives the one to one relationship
PharmacyName	NVARCHAR(120)	NOT NULL	Trading name, for example Mitford Pharma
LicenseNo	NVARCHAR(40)	UNIQUE, NOT NULL	DGDA drug licence number checked by the Super Admin
Area	NVARCHAR(60)	NOT NULL	Locality used by the customer's Area filter
Address	NVARCHAR(250)	NOT NULL	Full postal address printed on the invoice
ContactPhone	NVARCHAR(20)	NOT NULL	Shop landline or mobile
LogoPath	NVARCHAR(250)	NULL	Relative path to the shop logo image
CommissionRate	DECIMAL(5,2)	NOT NULL, CHECK 0-30	Platform commission percentage for this pharmacy
Status	NVARCHAR(15)	NOT NULL, CHECK	'Pending', 'Approved' or 'Suspended'
RegisteredAt	DATETIME2(0)	NOT NULL, DEFAULT	When the registration was submitted

4. Medicines – The products for sale. Two pharmacies selling the same brand are two separate rows with their own price and stock.

Column	Data Type	Constraint	Description
--------	-----------	------------	-------------

MedicineId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key
PharmacyId	INT	FOREIGN KEY, NOT NULL	References Pharmacies(PharmacyId); the isolation column
CategoryId	INT	FOREIGN KEY, NOT NULL	References Categories(CategoryId)
MedicineName	NVARCHAR(120)	NOT NULL	Brand name, for example Napa
GenericName	NVARCHAR(120)	NOT NULL	Molecule name, for example Paracetamol; searchable
Manufacturer	NVARCHAR(100)	NOT NULL	For example Beximco, Square, Renata
Strength	NVARCHAR(40)	NULL	For example 500mg or 100ml
UnitPrice	DECIMAL(10,2)	NOT NULL, CHECK > 0	Selling price per unit in taka
Stock	INT	NOT NULL, CHECK >= 0	Units currently on the shelf
MinStock	INT	NOT NULL, DEFAULT 10	Threshold that triggers the low stock alert
RequiresRx	BIT	NOT NULL, DEFAULT 0	1 means a prescription image is required at checkout
ExpiryDate	DATE	NOT NULL	Expiry date; expired stock is not offered to customers
Description	NVARCHAR(400)	NULL	Short description shown on the details form
ImagePath	NVARCHAR(250)	NULL	Relative path to the product image
IsActive	BIT	NOT NULL, DEFAULT 1	Set to 0 when the pharmacy is suspended or the item is delisted

5. Cart – The customer's live basket. One line per medicine, enforced by a UNIQUE constraint so adding twice updates the quantity.

Column	Data Type	Constraint	Description
CartId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key
CustomerId	INT	FOREIGN KEY, NOT NULL	References Users(UserId)
MedicineId	INT	FOREIGN KEY, NOT NULL	References Medicines(MedicineId)
Quantity	INT	NOT NULL, CHECK > 0	Units the customer intends to buy
AddedDate	DATETIME2(0)	NOT NULL, DEFAULT	Used to expire abandoned baskets
(composite)	UNIQUE	(CustomerId, MedicineId)	Stops duplicate lines for the same medicine

6. Orders – One row per completed checkout. A cart that spans two pharmacies becomes two orders. ItemsTotal and CommissionAmount are a snapshot of what was actually paid on the day, so a later price change cannot rewrite an old invoice.

Column	Data Type	Constraint	Description
OrderId	INT IDENTITY(1,1)	PRIMARY KEY	Invoice number shown to the customer
CustomerId	INT	FOREIGN KEY, NOT NULL	References Users(UserId)
PharmacyId	INT	FOREIGN KEY, NOT NULL	References Pharmacies(PharmacyId); one order per pharmacy
OrderDate	DATETIME2(0)	NOT NULL, DEFAULT	Timestamp used by every date range report
ItemsTotal	DECIMAL(12,2)	NOT NULL, CHECK >= 0	Sum of the order's line items after any offer discount
DeliveryCharge	DECIMAL(10,2)	NOT NULL, DEFAULT 60	Charged to the customer; the platform takes no commission on it
TotalAmount	COMPUTED PERSISTED	AS (ItemsTotal + DeliveryCharge)	The bill total, so it can never disagree with its parts
CommissionAmount	DECIMAL(12,2)	NOT NULL	ItemsTotal times the pharmacy's CommissionRate, frozen at checkout
DeliveryAddress	NVARCHAR(250)	NOT NULL	Copied from the profile but editable per order
PaymentMethod	NVARCHAR(20)	NOT NULL, CHECK	'CashOnDelivery', 'bKash', 'Nagad' or 'Card'
Status	NVARCHAR(15)	NOT NULL, CHECK	'Placed', 'Confirmed', 'Delivered' or 'Cancelled'

7. OrderItems – The mandatory junction table. It resolves the many to many relationship between Orders and Medicines.

Column	Data Type	Constraint	Description
OrderItemId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key
OrderId	INT	FOREIGN KEY, ON DELETE CASCADE	References Orders(OrderId)
MedicineId	INT	FOREIGN KEY, NOT NULL	References Medicines(MedicineId)
Quantity	INT	NOT NULL, CHECK > 0	Units of this medicine in this order
UnitPrice	DECIMAL(10,2)	NOT NULL, CHECK > 0	Price on the day of purchase, not today's price
Subtotal	COMPUTED PERSISTED	AS (Quantity * UnitPrice)	Derived by SQL Server, so it can never drift
(composite)	UNIQUE	(OrderId, MedicineId)	One line per medicine per order

8. Reviews – Ratings and comments. OrderId is carried so the application can prove the reviewer actually bought the item.

Column	Data Type	Constraint	Description
--------	-----------	------------	-------------

ReviewId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key
CustomerId	INT	FOREIGN KEY, NOT NULL	References Users(UserId)
MedicineId	INT	FOREIGN KEY, NOT NULL	References Medicines(MedicineId)
OrderId	INT	FOREIGN KEY, NOT NULL	References Orders(OrderId); makes the review verified
Rating	TINYINT	NOT NULL, CHECK BETWEEN 1 AND 5	Star rating
Comment	NVARCHAR(500)	NULL	Written feedback
ReviewDate	DATETIME2(0)	NOT NULL, DEFAULT	When the review was written
IsHidden	BIT	NOT NULL, DEFAULT 0	Set to 1 by Super Admin moderation instead of deleting
(composite)	UNIQUE	(CustomerId, MedicineId, OrderId)	One review per medicine per order

9. Offers – Percentage discount on one medicine, valid between two dates, created by the pharmacy owner.

Column	Data Type	Constraint	Description
OfferId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key
MedicineId	INT	FOREIGN KEY, NOT NULL	References Medicines(MedicineId)
OfferTitle	NVARCHAR(120)	NOT NULL	Shown on the customer's Offers screen
DiscountPercent	DECIMAL(5,2)	NOT NULL, CHECK 0 < x <= 70	Discount percentage
StartDate	DATE	NOT NULL	First day the offer applies
EndDate	DATE	NOT NULL, CHECK >= Start	Last day the offer applies
IsActive	BIT	NOT NULL, DEFAULT 1	Lets the owner pause an offer without deleting it

10. Prescriptions – Uploaded prescription image for an order containing a medicine whose RequiresRx flag is set.

Column	Data Type	Constraint	Description
PrescriptionId	INT IDENTITY(1,1)	PRIMARY KEY	Surrogate key
OrderId	INT	FOREIGN KEY, ON DELETE CASCADE	References Orders(OrderId)
CustomerId	INT	FOREIGN KEY, NOT NULL	References Users(UserId)

ImagePath	NVARCHAR(250)	NOT NULL	Relative path to the uploaded JPG or PNG
DoctorName	NVARCHAR(100)	NULL	Prescribing doctor as typed by the customer
UploadedAt	DATETIME2(0)	NOT NULL, DEFAULT	Upload timestamp
VerifyStatus	NVARCHAR(15)	NOT NULL, CHECK	'Pending', 'Approved' or 'Rejected' by the pharmacy

7. Database Query

DATABASE

```
CREATE DATABASE PharmaLinkDB;
GO
```

```
USE PharmaLinkDB;
GO
```

Users Table

```
CREATE TABLE Users (
    UserId          INT          IDENTITY(1,1) NOT NULL,
    FullName        NVARCHAR(100) NOT NULL,
    Email           NVARCHAR(120) NOT NULL,
    PasswordHash    NVARCHAR(200) NOT NULL,
    PasswordSalt    NVARCHAR(50)  NOT NULL,
    Phone           NVARCHAR(20)  NOT NULL,
    Address         NVARCHAR(250) NULL,
    UserType         NVARCHAR(15)  NOT NULL,
    Status          NVARCHAR(15)  NOT NULL CONSTRAINT DF_Users_Status DEFAULT
        ('Active'),
    CreatedAt       DATETIME2(0)  NOT NULL CONSTRAINT DF_Users_CreatedAt
        DEFAULT (SYSDATETIME()),

    CONSTRAINT PK_Users          PRIMARY KEY (UserId),
    CONSTRAINT UQ_Users_Email    UNIQUE (Email),
    CONSTRAINT UQ_Users_Phone    UNIQUE (Phone),
    CONSTRAINT CK_Users_Type     CHECK (UserType IN ('SuperAdmin', 'Admin',
        'Customer')),
    CONSTRAINT CK_Users_Status   CHECK (Status IN ('Pending', 'Active',
        'Suspended')),
    CONSTRAINT CK_Users_Email    CHECK (Email LIKE '%_@%._%')
);
GO
EXEC sp_columns 'Users';  -- to see the table structure
```

Categories Table

```

CREATE TABLE Categories (
    CategoryId INT IDENTITY(1,1) NOT NULL,
    CategoryName NVARCHAR(60) NOT NULL,
    Description NVARCHAR(200) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_Categories_IsActive
        DEFAULT (1),

    CONSTRAINT PK_Categories PRIMARY KEY (CategoryId),
    CONSTRAINT UQ_Categories_Name UNIQUE (CategoryName)
);
GO
EXEC sp_columns 'Categories'; -- to see the table structure

```

Pharmacies Table

```

CREATE TABLE Pharmacies (
    PharmacyId INT IDENTITY(1,1) NOT NULL,
    OwnerId INT NOT NULL,
    PharmacyName NVARCHAR(120) NOT NULL,
    LicenseNo NVARCHAR(40) NOT NULL,
    Area NVARCHAR(60) NOT NULL,
    Address NVARCHAR(250) NOT NULL,
    ContactPhone NVARCHAR(20) NOT NULL,
    LogoPath NVARCHAR(250) NULL,
    CommissionRate DECIMAL(5,2) NOT NULL CONSTRAINT DF_Pharmacies_Comm
        DEFAULT (8.00),
    Status NVARCHAR(15) NOT NULL CONSTRAINT
        DF_Pharmacies_Status DEFAULT ('Pending'),
    RegisteredAt DATETIME2(0) NOT NULL CONSTRAINT DF_Pharmacies_Reg
        DEFAULT (SYSDATETIME()),

    CONSTRAINT PK_Pharmacies PRIMARY KEY (PharmacyId),
    CONSTRAINT UQ_Pharmacies_Owner UNIQUE (OwnerId),
    CONSTRAINT UQ_Pharmacies_License UNIQUE (LicenseNo),
    CONSTRAINT FK_Pharmacies_Owner FOREIGN KEY (OwnerId) REFERENCES
        Users(UserId),
    CONSTRAINT CK_Pharmacies_Status CHECK (Status IN ('Pending',
        'Approved', 'Suspended')),
    CONSTRAINT CK_Pharmacies_Comm CHECK (CommissionRate >= 0 AND
        CommissionRate <= 30)
);
GO
EXEC sp_columns 'Pharmacies'; -- to see the table structure

```

Medicines Table

```

CREATE TABLE Medicines (
    MedicineId      INT          IDENTITY(1,1) NOT NULL,
    PharmacyId      INT          NOT NULL,
    CategoryId      INT          NOT NULL,
    MedicineName    NVARCHAR(120) NOT NULL,
    GenericName     NVARCHAR(120) NOT NULL,
    Manufacturer    NVARCHAR(100) NOT NULL,
    Strength        NVARCHAR(40)  NULL,
    UnitPrice       DECIMAL(10,2) NOT NULL,
    Stock           INT          NOT NULL CONSTRAINT DF_Medicines_Stock
                        DEFAULT (0),
    MinStock        INT          NOT NULL CONSTRAINT
                        DF_Medicines_MinStock DEFAULT (10),
    RequiresRx      BIT          NOT NULL CONSTRAINT DF_Medicines_Rx
                        DEFAULT (0),
    ExpiryDate      DATE          NOT NULL,
    Description     NVARCHAR(400) NULL,
    ImagePath       NVARCHAR(250) NULL,
    IsActive        BIT          NOT NULL CONSTRAINT
                        DF_Medicines_IsActive DEFAULT (1),

    CONSTRAINT PK_Medicines          PRIMARY KEY (MedicineId),
    CONSTRAINT FK_Medicines_Pharmacy FOREIGN KEY (PharmacyId) REFERENCES
        Pharmacies(PharmacyId),
    CONSTRAINT FK_Medicines_Category FOREIGN KEY (CategoryId) REFERENCES
        Categories(CategoryId),
    CONSTRAINT CK_Medicines_Price    CHECK (UnitPrice > 0),
    CONSTRAINT CK_Medicines_Stock    CHECK (Stock >= 0),
    CONSTRAINT CK_Medicines_MinStock CHECK (MinStock >= 0),
    CONSTRAINT UQ_Medicines_PerShop  UNIQUE (PharmacyId, MedicineName,
        Strength)
);
GO
EXEC sp_columns 'Medicines';  -- to see the table structure

```

Cart Table

```

CREATE TABLE Cart (
    CartId      INT          IDENTITY(1,1) NOT NULL,
    CustomerId  INT          NOT NULL,
    MedicineId  INT          NOT NULL,
    Quantity    INT          NOT NULL,
    AddedDate   DATETIME2(0) NOT NULL CONSTRAINT DF_Cart_AddedDate DEFAULT
        (SYSDATETIME()),

    CONSTRAINT PK_Cart          PRIMARY KEY (CartId),
    CONSTRAINT FK_Cart_Customer FOREIGN KEY (CustomerId) REFERENCES
        Users(UserId),
    CONSTRAINT FK_Cart_Medicine FOREIGN KEY (MedicineId) REFERENCES
        Medicines(MedicineId),
    CONSTRAINT UQ_Cart_Line     UNIQUE (CustomerId, MedicineId),
    CONSTRAINT CK_Cart_Qty     CHECK (Quantity > 0)
);
GO
EXEC sp_columns 'Cart';  -- to see the table structure

```

Orders Table

```
CREATE TABLE Orders (
    OrderId          INT          IDENTITY(1001,1) NOT NULL,    -- invoice
                        numbers start at 1001
    CustomerId       INT          NOT NULL,
    PharmacyId       INT          NOT NULL,
    OrderDate        DATETIME2(0) NOT NULL CONSTRAINT DF_Orders_Date
                        DEFAULT (SYSDATETIME()),
    ItemsTotal       DECIMAL(12,2) NOT NULL,
    DeliveryCharge   DECIMAL(10,2) NOT NULL CONSTRAINT DF_Orders_Delivery
                        DEFAULT (60.00),
    TotalAmount      AS (ItemsTotal + DeliveryCharge) PERSISTED,
    CommissionAmount DECIMAL(12,2) NOT NULL,
    DeliveryAddress  NVARCHAR(250) NOT NULL,
    PaymentMethod    NVARCHAR(20)  NOT NULL,
    Status           NVARCHAR(15)  NOT NULL CONSTRAINT DF_Orders_Status
                        DEFAULT ('Placed'),

    CONSTRAINT PK_Orders          PRIMARY KEY (OrderId),
    CONSTRAINT FK_Orders_Customer FOREIGN KEY (CustomerId) REFERENCES
        Users(UserId),
    CONSTRAINT FK_Orders_Pharmacy FOREIGN KEY (PharmacyId) REFERENCES
        Pharmacies(PharmacyId),
    CONSTRAINT CK_Orders_Items    CHECK (ItemsTotal >= 0),
    CONSTRAINT CK_Orders_Delivery CHECK (DeliveryCharge >= 0),
    CONSTRAINT CK_Orders_Payment CHECK (PaymentMethod IN
        ('CashOnDelivery', 'bKash', 'Nagad', 'Card')),
    CONSTRAINT CK_Orders_Status   CHECK (Status IN ('Placed', 'Confirmed',
        'Delivered', 'Cancelled'))
);
GO
EXEC sp_columns 'Orders';    -- to see the table structure
```

OrderItems Table

```
CREATE TABLE OrderItems (
    OrderItemId INT          IDENTITY(1,1) NOT NULL,
    OrderId     INT          NOT NULL,
    MedicineId  INT          NOT NULL,
    Quantity    INT          NOT NULL,
    UnitPrice   DECIMAL(10,2) NOT NULL,
    Subtotal    AS (Quantity * UnitPrice) PERSISTED,

    CONSTRAINT PK_OrderItems          PRIMARY KEY (OrderItemId),
    CONSTRAINT FK_OrderItems_Order    FOREIGN KEY (OrderId) REFERENCES
        Orders(OrderId) ON DELETE CASCADE,
    CONSTRAINT FK_OrderItems_Medicine FOREIGN KEY (MedicineId) REFERENCES
        Medicines(MedicineId),
    CONSTRAINT UQ_OrderItems_Line     UNIQUE (OrderId, MedicineId),
    CONSTRAINT CK_OrderItems_Qty      CHECK (Quantity > 0),
    CONSTRAINT CK_OrderItems_Price    CHECK (UnitPrice > 0)
);
GO
EXEC sp_columns 'OrderItems';    -- to see the table structure
```

Reviews Table

```
CREATE TABLE Reviews (
    ReviewId      INT          IDENTITY(1,1) NOT NULL,
    CustomerId    INT          NOT NULL,
    MedicineId     INT          NOT NULL,
    OrderId       INT          NOT NULL,
    Rating        TINYINT      NOT NULL,
    Comment       NVARCHAR(500) NULL,
    ReviewDate    DATETIME2(0) NOT NULL CONSTRAINT DF_Reviews_Date DEFAULT
        (SYSDATETIME()),
    IsHidden      BIT          NOT NULL CONSTRAINT DF_Reviews_Hidden DEFAULT
        (0),

    CONSTRAINT PK_Reviews          PRIMARY KEY (ReviewId),
    CONSTRAINT FK_Reviews_Customer FOREIGN KEY (CustomerId) REFERENCES
        Users(UserId),
    CONSTRAINT FK_Reviews_Medicine FOREIGN KEY (MedicineId) REFERENCES
        Medicines(MedicineId),
    CONSTRAINT FK_Reviews_Order    FOREIGN KEY (OrderId)    REFERENCES
        Orders(OrderId),
    CONSTRAINT UQ_Reviews_OneEach  UNIQUE (CustomerId, MedicineId,
        OrderId),
    CONSTRAINT CK_Reviews_Rating   CHECK (Rating BETWEEN 1 AND 5)
);
GO
EXEC sp_columns 'Reviews';  -- to see the table structure
```

Offers Table

```
CREATE TABLE Offers (
    OfferId      INT          IDENTITY(1,1) NOT NULL,
    MedicineId   INT          NOT NULL,
    OfferTitle    NVARCHAR(120) NOT NULL,
    DiscountPercent DECIMAL(5,2) NOT NULL,
    StartDate     DATE        NOT NULL,
    EndDate       DATE        NOT NULL,
    IsActive      BIT          NOT NULL CONSTRAINT DF_Offers_IsActive
        DEFAULT (1),

    CONSTRAINT PK_Offers          PRIMARY KEY (OfferId),
    CONSTRAINT FK_Offers_Medicine FOREIGN KEY (MedicineId) REFERENCES
        Medicines(MedicineId),
    CONSTRAINT CK_Offers_Percent  CHECK (DiscountPercent > 0 AND
        DiscountPercent <= 70),
    CONSTRAINT CK_Offers_Dates    CHECK (EndDate >= StartDate)
);
GO
EXEC sp_columns 'Offers';  -- to see the table structure
```

Prescriptions Table

```

CREATE TABLE Prescriptions (
    PrescriptionId INT          IDENTITY(1,1) NOT NULL,
    OrderId        INT          NOT NULL,
    CustomerId     INT          NOT NULL,
    ImagePath      NVARCHAR(250) NOT NULL,
    DoctorName     NVARCHAR(100) NULL,
    UploadedAt     DATETIME2(0) NOT NULL CONSTRAINT DF_Rx_Uploaded DEFAULT
        (SYSDATETIME()),
    VerifyStatus   NVARCHAR(15) NOT NULL CONSTRAINT DF_Rx_Status DEFAULT
        ('Pending'),

    CONSTRAINT PK_Prescriptions          PRIMARY KEY (PrescriptionId),
    CONSTRAINT FK_Prescriptions_Order    FOREIGN KEY (OrderId)
        REFERENCES Orders(OrderId) ON DELETE CASCADE,
    CONSTRAINT FK_Prescriptions_Customer FOREIGN KEY (CustomerId)
        REFERENCES Users(UserId),
    CONSTRAINT CK_Prescriptions_Status   CHECK (VerifyStatus IN ('Pending',
        'Approved', 'Rejected'))
);
GO
EXEC sp_columns 'Prescriptions';  -- to see the table structure

```

7.1 Sample Data

The script inserts at least three rows into every table, including one Super Admin, four pharmacy owners and four customers, so that every query below returns a meaningful result the moment the script finishes. The complete INSERT section is in `database/schema.sql`.

7.2 Feature Queries

Each query sits behind a named form and a numbered requirement. In the application every @parameter is supplied through SqlCommand.Parameters and nothing is concatenated into a query string, which is what keeps the system safe from SQL injection. JOIN, GROUP BY, HAVING and the aggregates SUM, AVG and COUNT are each used more than once.

7.1 Login and role routing

Form: LoginForm

```

DECLARE @Email NVARCHAR(120) = 'rahim@gmail.com', @PasswordHash
        NVARCHAR(200) = 'HASH_Cust@123';

SELECT  u.UserId, u.FullName, u.UserType, u.Status,
        p.PharmacyId                -- NULL for SuperAdmin and
            Customer
FROM    Users u
        LEFT JOIN Pharmacies p ON p.OwnerId = u.UserId
WHERE   u.Email      = @Email
        AND u.PasswordHash = @PasswordHash
        AND u.Status  = 'Active';

```

This one query does the whole of authentication. It returns the user id, name, user type and, through a LEFT JOIN on Pharmacies, the PharmacyId when the user happens to be a pharmacy owner. That PharmacyId is stored in the session and every later Admin query filters on it. Because

the WHERE clause also demands Status = 'Active', a Pending or Suspended account is refused by the same statement that checks the password, with no second round trip.

7.2 Filter medicines by price range

Form: CustomerHomeForm

```
DECLARE @MinPrice DECIMAL(10,2) = 0, @MaxPrice DECIMAL(10,2) = 50;

SELECT  m.MedicineId, m.MedicineName, m.Strength, m.UnitPrice, m.Stock,
        ph.PharmacyName
FROM    Medicines m
        INNER JOIN Pharmacies ph ON ph.PharmacyId = m.PharmacyId
WHERE   m.IsActive = 1
        AND ph.Status = 'Approved'
        AND m.UnitPrice BETWEEN @MinPrice AND @MaxPrice
ORDER BY m.UnitPrice ASC;
```

The price range ComboBox supplies @MinPrice and @MaxPrice. The join to Pharmacies is not decoration: it lets the WHERE clause exclude any pharmacy that is not Approved, so a suspended shop's stock disappears from the catalogue without a single row being deleted.

7.3 Filter by category and availability

Form: CustomerHomeForm

```
DECLARE @CategoryId INT = 2;  -- Painkiller

SELECT  m.MedicineId, m.MedicineName, c.CategoryName, m.UnitPrice, m.Stock,
        ph.PharmacyName, ph.Area
FROM    Medicines m
        INNER JOIN Categories c ON c.CategoryId = m.CategoryId
        INNER JOIN Pharmacies ph ON ph.PharmacyId = m.PharmacyId
WHERE   c.CategoryId = @CategoryId
        AND m.Stock > 0
        AND m.IsActive = 1
        AND ph.Status = 'Approved'
ORDER BY ph.Area, m.MedicineName;
```

Two filters combined in one statement. Stock > 0 backs the In stock only dropdown, and the second join brings in the category name so the grid can show it without a second query. Results are ordered by area so that customers see nearby pharmacies grouped together.

7.4 Keyword search

Form: shared search box on every dashboard

```

DECLARE @Keyword NVARCHAR(60) = 'paracetamol';

SELECT  m.MedicineId, m.MedicineName, m.GenericName, m.Manufacturer,
        m.UnitPrice, m.Stock, ph.PharmacyName
FROM    Medicines m
        INNER JOIN Pharmacies ph ON ph.PharmacyId = m.PharmacyId
WHERE   m.IsActive = 1
        AND (m.MedicineName LIKE '%' + @Keyword + '%'
            OR m.GenericName LIKE '%' + @Keyword + '%'
            OR m.Manufacturer LIKE '%' + @Keyword + '%')
ORDER BY m.MedicineName;

```

The search deliberately covers three columns. A customer who types 'paracetamol' does not know that the brand is called Napa, and a customer who types 'Square' wants everything that manufacturer makes. Searching the generic name is what makes the catalogue useful to someone holding a doctor's chit rather than a box.

7.5 Cart: add, remove and view with total

Form: MedicineDetailsForm and CartForm

```

DECLARE @CustomerId INT = 6, @MedicineId INT = 1, @Quantity INT = 30;

-- add an item (insert a new line, or increase the quantity if the line
    already exists)
MERGE Cart AS target
USING (SELECT @CustomerId AS CustomerId, @MedicineId AS MedicineId,
    @Quantity AS Quantity) AS source
    ON target.CustomerId = source.CustomerId
    AND target.MedicineId = source.MedicineId
WHEN MATCHED THEN
    UPDATE SET target.Quantity = target.Quantity + source.Quantity
WHEN NOT MATCHED THEN
    INSERT (CustomerId, MedicineId, Quantity)
    VALUES (source.CustomerId, source.MedicineId, source.Quantity);

-- change the quantity of one line, or remove it
UPDATE Cart SET Quantity = @Quantity WHERE CustomerId = @CustomerId AND
    MedicineId = @MedicineId;
DELETE FROM Cart WHERE CustomerId = @CustomerId AND MedicineId =
    @MedicineId;

-- view the basket, with today's offer applied to each line
SELECT ph.PharmacyName,
    m.MedicineName,
    ct.Quantity,
    m.UnitPrice AS ListPrice,
    ISNULL(d.Pct, 0) AS DiscountPercent,
    CAST(m.UnitPrice * (1 - ISNULL(d.Pct,0)/100.0) AS DECIMAL(10,2)) AS
        PriceYouPay,
    CAST(ct.Quantity * m.UnitPrice * (1 - ISNULL(d.Pct,0)/100.0) AS
        DECIMAL(12,2)) AS LineTotal
FROM Cart ct
    INNER JOIN Medicines m ON m.MedicineId = ct.MedicineId
    INNER JOIN Pharmacies ph ON ph.PharmacyId = m.PharmacyId
    OUTER APPLY (SELECT MAX(ofr.DiscountPercent) AS Pct
        FROM Offers ofr
        WHERE ofr.MedicineId = m.MedicineId
            AND ofr.IsActive = 1
            AND CAST(GETDATE() AS DATE) BETWEEN ofr.StartDate
                AND ofr.EndDate) d
WHERE ct.CustomerId = @CustomerId
ORDER BY ph.PharmacyName, m.MedicineName;

-- basket summary per pharmacy: this is how many orders checkout will create
SELECT ph.PharmacyId,
    ph.PharmacyName,
    COUNT(*)
        AS Lines,
    CAST(SUM(ct.Quantity * m.UnitPrice) AS DECIMAL(12,2))
        AS BeforeDiscount,
    CAST(SUM(ct.Quantity * m.UnitPrice * (1 - ISNULL(d.Pct,0)/100.0))
        AS DECIMAL(12,2)) AS ItemsTotal,
    60.00
        AS DeliveryCharge

```

```

FROM    Cart ct
        INNER JOIN Medicines m ON m.MedicineId = ct.MedicineId
        INNER JOIN Pharmacies ph ON ph.PharmacyId = m.PharmacyId
        OUTER APPLY (SELECT MAX(ofr.DiscountPercent) AS Pct
                     FROM    Offers ofr
                     WHERE   ofr.MedicineId = m.MedicineId AND ofr.IsActive
                             = 1
                             AND CAST(GETDATE() AS DATE) BETWEEN ofr.StartDate
                                AND ofr.EndDate) d
WHERE    ct.CustomerId = @CustomerId
GROUP BY ph.PharmacyId, ph.PharmacyName
ORDER BY ph.PharmacyName;

```

MERGE handles the add case in one statement: if the customer already has that medicine in the basket the quantity is increased, otherwise a new line is inserted. That is what keeps the UNIQUE constraint on (CustomerId, MedicineId) from ever being violated. The view query is offer aware: OUTER APPLY finds the best discount running today for each medicine, so the price the customer sees in the cart is the same price the checkout will charge. The second SELECT groups the basket by pharmacy, because a cart that spans two pharmacies becomes two separate orders at checkout and each one carries its own delivery charge.

7.6 Checkout: order, line items and stock in one transaction

Form: CheckoutForm

```

DECLARE @CustomerId      INT          = 6;      -- from the session
DECLARE @PharmacyId      INT          = 1;      -- checkout runs once per
        pharmacy in the cart
DECLARE @DeliveryCharge  DECIMAL(10,2) = 60.00; -- Tk 60 per pharmacy
DECLARE @DeliveryAddress NVARCHAR(250) = 'House 7, Road 3, Mirpur 10,
        Dhaka';
DECLARE @PaymentMethod   NVARCHAR(20)  = 'bKash';
DECLARE @NewOrderId INT, @Total DECIMAL(12,2), @CommRate DECIMAL(5,2);

BEGIN TRANSACTION;

-- 1. what this pharmacy's slice of the basket costs, with today's
    offers applied
SELECT @Total = CAST(SUM(ct.Quantity * m.UnitPrice * (1 -
        ISNULL(d.Pct,0)/100.0)) AS DECIMAL(12,2))
FROM      Cart ct
        INNER JOIN Medicines m ON m.MedicineId = ct.MedicineId
        OUTER APPLY (SELECT MAX(ofr.DiscountPercent) AS Pct FROM Offers
        ofr
                WHERE ofr.MedicineId = m.MedicineId AND
                        ofr.IsActive = 1
                        AND CAST(GETDATE() AS DATE) BETWEEN ofr.StartDate
                        AND ofr.EndDate) d
WHERE     ct.CustomerId = @CustomerId AND m.PharmacyId = @PharmacyId;

SELECT @CommRate = CommissionRate FROM Pharmacies WHERE PharmacyId =
        @PharmacyId;

-- 2. the order header, with the commission frozen at today's rate
INSERT INTO Orders (CustomerId, PharmacyId, ItemsTotal, DeliveryCharge,
        CommissionAmount,
                DeliveryAddress, PaymentMethod, Status)
VALUES (@CustomerId, @PharmacyId, @Total, @DeliveryCharge,
        CAST(@Total * @CommRate / 100.0 AS DECIMAL(12,2)),
        @DeliveryAddress, @PaymentMethod, 'Placed');

SET @NewOrderId = SCOPE_IDENTITY();

-- 3. one line per cart row, at the discounted price the customer
    actually saw
INSERT INTO OrderItems (OrderId, MedicineId, Quantity, UnitPrice)
SELECT @NewOrderId, ct.MedicineId, ct.Quantity,
        CAST(m.UnitPrice * (1 - ISNULL(d.Pct,0)/100.0) AS DECIMAL(10,2))
FROM      Cart ct
        INNER JOIN Medicines m ON m.MedicineId = ct.MedicineId
        OUTER APPLY (SELECT MAX(ofr.DiscountPercent) AS Pct FROM Offers
        ofr
                WHERE ofr.MedicineId = m.MedicineId AND
                        ofr.IsActive = 1
                        AND CAST(GETDATE() AS DATE) BETWEEN ofr.StartDate
                        AND ofr.EndDate) d
WHERE     ct.CustomerId = @CustomerId AND m.PharmacyId = @PharmacyId;

-- 4. take the stock off the shelf

```

```

UPDATE m SET m.Stock = m.Stock - ct.Quantity
FROM   Medicines m INNER JOIN Cart ct ON ct.MedicineId = m.MedicineId
WHERE  ct.CustomerId = @CustomerId AND m.PharmacyId = @PharmacyId;

-- 5. clear only this pharmacy's lines; the rest of the basket becomes
      the next order
DELETE ct
FROM   Cart ct INNER JOIN Medicines m ON m.MedicineId = ct.MedicineId
WHERE  ct.CustomerId = @CustomerId AND m.PharmacyId = @PharmacyId;

COMMIT TRANSACTION;

```

This is the most important query group in the system. Note the @PharmacyId filter on all four cart statements: the customer's basket may hold medicines from two pharmacies, and each pharmacy becomes its own order with its own delivery charge, so checkout runs this batch once per pharmacy in the cart. Five things must then happen together: the order header is written, one line is written per cart row at the price the customer was shown, the stock of every medicine is reduced, that pharmacy's cart lines are cleared, and the commission is frozen on the order row. If any one of them failed on its own the database would be left with an order that has no items, or stock that was sold twice. Wrapping them in a single transaction is what makes the checkout safe.

7.7 Pharmacy earnings (JOIN + GROUP BY + SUM)

Form: SuperAdminSalesReportForm and AdminEarningsForm

```

SELECT  ph.PharmacyId, ph.PharmacyName, ph.Area,
        COUNT(DISTINCT o.OrderId) AS TotalOrders,
        SUM(oi.Quantity)          AS UnitsSold,
        SUM(oi.Subtotal)          AS GrossSales,
        CAST(SUM(oi.Subtotal) * ph.CommissionRate / 100.0 AS DECIMAL(12,2))
        AS PlatformCommission,
        CAST(AVG(oi.UnitPrice) AS DECIMAL(10,2))
        AS AverageItemPrice
FROM    Pharmacies ph
        INNER JOIN Orders      o  ON o.PharmacyId = ph.PharmacyId
        INNER JOIN OrderItems oi ON oi.OrderId   = o.OrderId
WHERE   o.Status <> 'Cancelled'
GROUP BY ph.PharmacyId, ph.PharmacyName, ph.Area, ph.CommissionRate
ORDER BY GrossSales DESC;

```

The Super Admin runs this as it stands to see every pharmacy. The pharmacy owner runs the same query with an added WHERE ph.PharmacyId = @PharmacyId and sees only his own row, which is the clearest possible demonstration of data isolation: one query, two role scopes. Commission is recomputed from the pharmacy rate rather than summed from Orders, because the join to OrderItems multiplies the order rows and would inflate a plain SUM of the header column.

7.8 Low stock alert

Form: AdminInventoryForm

```

DECLARE @PharmacyId INT = 3;    -- Lazz Care Pharmacy, from the session

SELECT  m.MedicineId, m.MedicineName, m.Strength, c.CategoryName,
        m.Stock, m.MinStock, (m.MinStock - m.Stock) AS ShortfallUnits
FROM    Medicines m
        INNER JOIN Categories c ON c.CategoryId = m.CategoryId
WHERE   m.PharmacyId = @PharmacyId    -- data isolation: own pharmacy
        only
        AND m.Stock < m.MinStock
        AND m.IsActive = 1
ORDER BY ShortfallUnits DESC;

```

Comparing two columns of the same row is what makes this alert useful. A fixed threshold would be wrong, because ten boxes of a glucometer is plenty while ten strips of Napa is nothing. The shortfall column tells the owner how many units to order. The WHERE clause carries the PharmacyId, so the alert can never leak another shop's inventory.

7.9 Reviews for one medicine (JOIN)

Form: MedicineDetailsForm

```

DECLARE @MedicineId INT = 1;    -- Napa 500mg

SELECT  r.ReviewId, u.FullName AS ReviewerName, r.Rating, r.Comment,
        r.ReviewDate
FROM    Reviews r
        INNER JOIN Users u ON u.UserId = r.CustomerId
        INNER JOIN Medicines m ON m.MedicineId = r.MedicineId
WHERE   r.MedicineId = @MedicineId
        AND r.IsHidden = 0
ORDER BY r.ReviewDate DESC;

```

Reviews store only a CustomerId, so the join to Users is what turns a number into the reviewer's name on screen. Hidden reviews are excluded here rather than deleted at source, which means Super Admin moderation takes effect immediately on every customer screen while the row survives for audit.

7.10 Pharmacies rated below 2.5 (JOIN + GROUP BY + HAVING + AVG)

Form: SuperAdminLowRatedShopsForm

```

SELECT  ph.PharmacyId, ph.PharmacyName, ph.Area,
        u.FullName AS OwnerName, u.Phone AS OwnerPhone,
        COUNT(r.ReviewId) AS TotalReviews,
        CAST(AVG(CAST(r.Rating AS DECIMAL(4,2))) AS DECIMAL(4,2)) AS
            AverageRating
FROM    Pharmacies ph
        INNER JOIN Users u ON u.UserId = ph.OwnerId
        INNER JOIN Medicines m ON m.PharmacyId = ph.PharmacyId
        INNER JOIN Reviews r ON r.MedicineId = m.MedicineId
WHERE   r.IsHidden = 0
GROUP BY ph.PharmacyId, ph.PharmacyName, ph.Area, u.FullName, u.Phone
HAVING  AVG(CAST(r.Rating AS DECIMAL(4,2))) < 2.5
        AND COUNT(r.ReviewId) >= 2
ORDER BY AverageRating ASC;

```

Ratings sit on medicines, not on pharmacies, so the average has to be built by joining three tables and grouping back up to the pharmacy. HAVING is the right clause rather than WHERE because the condition is on the aggregate itself. The second HAVING condition, COUNT(ReviewId) >= 2, is a deliberate fairness rule: one angry customer should not be enough to put a shop on the suspension list.

7.11 Super Admin: approve and suspend a pharmacy owner

Form: SuperAdminManageShopsForm

```
DECLARE @PharmacyId INT = 4, @OwnerId INT = 5;    -- New Life Pharmacy,
                                                still Pending

-- approve a pending pharmacy owner
UPDATE Pharmacies SET Status = 'Approved' WHERE PharmacyId = @PharmacyId;
UPDATE Users      SET Status = 'Active'   WHERE UserId      = @OwnerId;

-- suspend an owner and hide their medicines from customers
UPDATE Pharmacies SET Status = 'Suspended' WHERE PharmacyId = @PharmacyId;
UPDATE Users      SET Status = 'Suspended' WHERE UserId      = @OwnerId;
UPDATE Medicines  SET IsActive = 0        WHERE PharmacyId = @PharmacyId;
```

Approval flips two rows, because the pharmacy record and the login account are separate concerns. Suspension flips three: the pharmacy, the account and every medicine that pharmacy lists. Nothing is deleted anywhere, so the invoices customers already hold and the sales figures in last month's report stay exactly as they were.

7.12 Active offers today with the discounted price

Form: CustomerOffersForm

```
SELECT  o.OfferId, o.OfferTitle, m.MedicineName, m.Strength,
        ph.PharmacyName,
        m.UnitPrice AS OriginalPrice, o.DiscountPercent,
        CAST(m.UnitPrice * (1 - o.DiscountPercent / 100.0) AS
            DECIMAL(10,2)) AS DiscountedPrice,
        o.EndDate
FROM    Offers o
        INNER JOIN Medicines m ON m.MedicineId = o.MedicineId
        INNER JOIN Pharmacies ph ON ph.PharmacyId = m.PharmacyId
WHERE   o.IsActive = 1
        AND CAST(GETDATE() AS DATE) BETWEEN o.StartDate AND o.EndDate
        AND m.Stock > 0
        AND ph.Status = 'Approved'
ORDER BY o.DiscountPercent DESC;
```

The discounted price is calculated in the query rather than in C#, so the same number appears on the offers screen, the details screen and the cart without three chances to disagree. Filtering on the date range inside the query means an expired offer can never be shown by mistake, whatever the form does.

7.13 Revenue by area (JOIN + GROUP BY + HAVING + COUNT + SUM)

Form: SuperAdminDashboard tiles


```

SELECT  ph.Area,
        COUNT(DISTINCT ph.PharmacyId) AS PharmaciesInArea,
        COUNT(DISTINCT o.OrderId)     AS Orders,
        SUM(o.TotalAmount)             AS Revenue,
        SUM(o.CommissionAmount)        AS CommissionEarned
FROM    Pharmacies ph
        INNER JOIN Orders o ON o.PharmacyId = ph.PharmacyId
WHERE   o.Status = 'Delivered'
GROUP BY ph.Area
HAVING  SUM(o.TotalAmount) > 100
ORDER BY Revenue DESC;

```

An extra reporting query that answers a question the Super Admin actually asks: which parts of the city are worth expanding into. It groups delivered orders by pharmacy area and uses HAVING to drop areas that have not yet crossed a meaningful revenue figure.

7.14 Sign up: register a customer or a pharmacy owner

Form: SignUpForm

```

DECLARE @NewUserId INT;

-- a customer signs up: usable straight away
INSERT INTO Users (FullName, Email, PasswordHash, PasswordSalt, Phone,
                  Address, UserType, Status)
VALUES (N'Tanjila Akter', 'tanjila@gmail.com', 'HASH_Cust@123', 'SA1',
        '01811000010',
        N'Mohammadpur, Dhaka', 'Customer', 'Active');

-- a pharmacy owner signs up: the account and the pharmacy both start
    Pending
INSERT INTO Users (FullName, Email, PasswordHash, PasswordSalt, Phone,
                  Address, UserType, Status)
VALUES (N'Imran Hossain', 'popular@pharmalink.com.bd', 'HASH_Pharma@123',
        'SA2', '01711000011',
        N'Panthapath, Dhaka', 'Admin', 'Pending');
SET @NewUserId = SCOPE_IDENTITY();

INSERT INTO Pharmacies (OwnerId, PharmacyName, LicenseNo, Area, Address,
                       ContactPhone, Status)
VALUES (@NewUserId, N'Popular Pharmacy', 'DGDA-DH-10099', N'Panthapath',
        N'House 9, Panthapath, Dhaka 1205', '02955000099', 'Pending');

```

A customer is created Active and can use the platform immediately. A pharmacy owner is created Pending together with a Pending pharmacy row, and neither becomes usable until the Super Admin approves it, which is the workflow behind query 11. The UNIQUE constraints on Email and Phone are what actually stop a duplicate account, so the form checks first only to give a friendly message.

7.15 Medicine CRUD for the logged in pharmacy

Form: AdminMedicineForm and the Add / Edit Medicine dialog

```

DECLARE @PharmacyId INT = 1, @MedicineId INT = 3;

-- CREATE
INSERT INTO Medicines (PharmacyId, CategoryId, MedicineName, GenericName,
    Manufacturer,
        Strength, UnitPrice, Stock, MinStock, RequiresRx,
        ExpiryDate, Description)
VALUES (@PharmacyId, 7, N'Seclo', N'Omeprazole', N'Square Pharmaceuticals',
    N'20mg', 7.00, 120, 30, 1, '2027-12-31', N'Proton pump inhibitor
    for acidity and ulcer');

-- READ (the DataGridView)
SELECT m.MedicineId, m.MedicineName, m.GenericName, c.CategoryName,
    m.Manufacturer,
    m.Strength, m.UnitPrice, m.Stock, m.MinStock, m.RequiresRx,
        m.ExpiryDate
FROM Medicines m INNER JOIN Categories c ON c.CategoryId = m.CategoryId
WHERE m.PharmacyId = @PharmacyId AND m.IsActive = 1
ORDER BY m.MedicineName;

-- UPDATE
UPDATE Medicines
SET UnitPrice = 9.00, Stock = 320, MinStock = 60, ExpiryDate =
    '2028-06-30'
WHERE MedicineId = @MedicineId AND PharmacyId = @PharmacyId;

-- DELETE (soft, so OrderItems rows stay valid)
UPDATE Medicines SET IsActive = 0
WHERE MedicineId = @MedicineId AND PharmacyId = @PharmacyId;

-- create and pause a discount offer on one of my own medicines
    (requirement 14)
INSERT INTO Offers (MedicineId, OfferTitle, DiscountPercent, StartDate,
    EndDate)
SELECT m.MedicineId, N'Winter Cold Pack - 12% off', 12.00, '2026-11-01',
    '2026-11-30'
FROM Medicines m
WHERE m.MedicineId = @MedicineId AND m.PharmacyId = @PharmacyId; -- own
    medicines only

UPDATE ofr SET ofr.IsActive = 0
FROM Offers ofr INNER JOIN Medicines m ON m.MedicineId = ofr.MedicineId
WHERE ofr.OfferId = 4 AND m.PharmacyId = @PharmacyId;

```

The statements behind the Add, Update, Delete and Create Offer buttons. Every one of them carries PharmacyId, so an Admin cannot create a medicine under someone else's shop, and cannot update or delete a row that is not his. Delete is a soft delete: setting IsActive to 0 keeps the foreign keys from OrderItems intact, so old invoices still resolve.

7.16 Super Admin: user list, category CRUD and commission rate

Form: SuperAdminManageUsersForm, ManageCategoriesForm

```

DECLARE @Keyword NVARCHAR(60) = '', @Status NVARCHAR(15) = '', @CategoryId
        INT = 6, @PharmacyId INT = 1;

-- all users, with the pharmacy name for owners (requirement 4)
SELECT  u.UserId, u.FullName, u.Email, u.Phone, u.UserType, u.Status,
        ph.PharmacyName, u.CreatedAt
FROM    Users u
        LEFT JOIN Pharmacies ph ON ph.OwnerId = u.UserId
WHERE   (@Keyword = '' OR u.FullName LIKE '%' + @Keyword + '%' OR u.Email
        LIKE '%' + @Keyword + '%')
        AND (@Status = '' OR u.Status = @Status)
ORDER BY u.UserType, u.FullName;

-- category master list (requirement 7)
INSERT INTO Categories (CategoryName, Description) VALUES (N'Skin Care',
        N'Dermatological creams and ointments');
UPDATE Categories SET Description = N'Analgesic, antipyretic and
        anti-inflammatory medicines' WHERE CategoryId = 2;
UPDATE Categories SET IsActive = 0 WHERE CategoryId = @CategoryId; --
        deactivate, never delete

-- set one pharmacy's commission rate (requirement 9)
UPDATE Pharmacies SET CommissionRate = 9.50 WHERE PharmacyId = @PharmacyId;

-- the moderation queue, and hiding an abusive review (requirement 8)
SELECT  r.ReviewId, u.FullName AS Reviewer, m.MedicineName, ph.PharmacyName,
        r.Rating, r.Comment, r.ReviewDate, r.OrderId
FROM    Reviews r
        INNER JOIN Users      u  ON u.UserId      = r.CustomerId
        INNER JOIN Medicines  m  ON m.MedicineId  = r.MedicineId
        INNER JOIN Pharmacies ph ON ph.PharmacyId = m.PharmacyId
WHERE   r.Rating <= 2 AND r.IsHidden = 0
ORDER BY r.ReviewDate DESC;

UPDATE Reviews SET IsHidden = 1 WHERE ReviewId = 5; -- hidden, never
        deleted

```

The small administrative statements that the requirements list but that have no report of their own. The user list joins Users to Pharmacies so that a pharmacy owner's shop name appears beside his name, and it accepts a keyword and a status from the search box and the Status ComboBox. Categories are deactivated rather than deleted, because Medicines rows point at them.

7.17 Customer: place a review, and read the order history

Form: GiveRatingForm and OrderHistoryForm

```

DECLARE @CustomerId INT = 6, @MedicineId INT = 3, @OrderId INT = 1002,
        @Rating TINYINT = 4;

-- write a review, but only for a delivered order that actually contained
    the medicine
INSERT INTO Reviews (CustomerId, MedicineId, OrderId, Rating, Comment)
SELECT @CustomerId, @MedicineId, @OrderId, @Rating, N'Sealed pack,
        delivered on time.'
WHERE EXISTS (SELECT 1
              FROM   OrderItems oi INNER JOIN Orders o ON o.OrderId =
                    oi.OrderId
              WHERE  oi.OrderId    = @OrderId
                    AND oi.MedicineId = @MedicineId
                    AND o.CustomerId = @CustomerId
                    AND o.Status     = 'Delivered');

-- order history with item count and the reviewable flag (requirement 27)
SELECT  o.OrderId, o.OrderDate, ph.PharmacyName,
        COUNT(oi.OrderItemId) AS Items,
        o.ItemsTotal, o.DeliveryCharge, o.TotalAmount,
        o.PaymentMethod, o.Status,
        CASE WHEN o.Status = 'Delivered'
              AND EXISTS (SELECT 1 FROM OrderItems x
                        WHERE x.OrderId = o.OrderId
                        AND NOT EXISTS (SELECT 1 FROM Reviews r
                                WHERE r.OrderId = o.OrderId AND
                                      r.MedicineId =
                                          x.MedicineId))
              THEN 1 ELSE 0 END AS CanReview
FROM    Orders o
        INNER JOIN Pharmacies ph ON ph.PharmacyId = o.PharmacyId
        INNER JOIN OrderItems oi ON oi.OrderId    = o.OrderId
WHERE   o.CustomerId = @CustomerId
GROUP BY o.OrderId, o.OrderDate, ph.PharmacyName, o.ItemsTotal,
        o.DeliveryCharge,
        o.TotalAmount, o.PaymentMethod, o.Status
ORDER BY o.OrderDate DESC;

```

The review INSERT is guarded twice. The WHERE EXISTS clause proves the customer really bought that medicine on that order, and the UNIQUE constraint on (CustomerId, MedicineId, OrderId) stops the same purchase being rated twice, so a duplicate attempt fails at the database even if the form is bypassed. The order history query below it is what fills the customer's Orders grid, including the invoice total from the computed TotalAmount column.

7.18 Profile update, password change and prescription verification

Form: MyProfileForm and the Verify Prescription dialog

```

DECLARE @UserId INT = 6, @OldHash NVARCHAR(200) = 'HASH_Cust@123', @NewHash
        NVARCHAR(200) = 'HASH_Cust@456';
DECLARE @PrescriptionId INT = 3, @OrderId INT = 1005, @PharmacyId INT = 3;

-- update own profile (requirements 17, 30)
UPDATE Users
SET     FullName = N'Rahim Uddin', Phone = '01811000006', Address = N'House
        7, Road 3, Mirpur 10, Dhaka'
WHERE   UserId = @UserId;

-- change own password: succeeds only if the current password hash matches
UPDATE Users
SET     PasswordHash = @NewHash, PasswordSalt = 'NEWSALT'
WHERE   UserId = @UserId AND PasswordHash = @OldHash;

-- the customer uploads a prescription during checkout (requirement 26)
INSERT INTO Prescriptions (OrderId, CustomerId, ImagePath, DoctorName)
VALUES (1005, 9, 'rx/2026/08/rx-1005b.jpg', N'Dr. Anisur Rahman, MBBS');

-- the pharmacy's prescription queue (requirement 16)
SELECT  p.PrescriptionId, p.OrderId, u.FullName AS Customer, p.DoctorName,
        p.ImagePath, p.VerifyStatus
FROM    Prescriptions p
        INNER JOIN Orders o ON o.OrderId      = p.OrderId
        INNER JOIN Users  u ON u.UserId       = p.CustomerId
WHERE   o.PharmacyId = @PharmacyId AND p.VerifyStatus = 'Pending';

UPDATE Prescriptions SET VerifyStatus = 'Approved' WHERE PrescriptionId =
        @PrescriptionId;

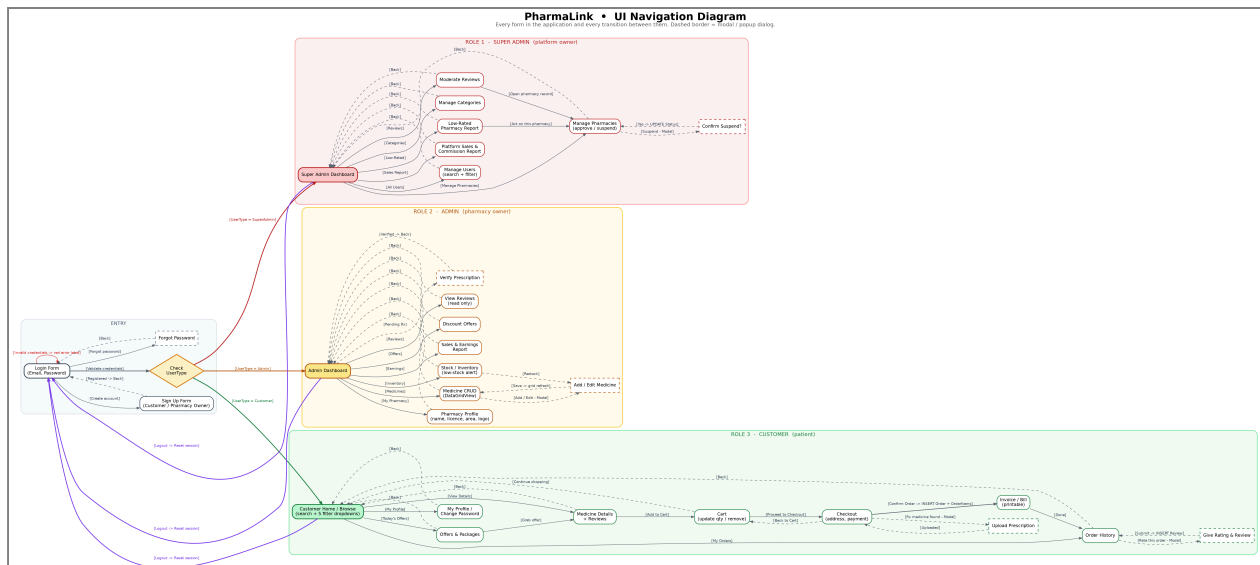
-- an order cannot be confirmed while a prescription on it is still Pending
UPDATE Orders SET Status = 'Confirmed'
WHERE   OrderId = @OrderId
        AND NOT EXISTS (SELECT 1 FROM Prescriptions p
                        WHERE p.OrderId = @OrderId AND p.VerifyStatus <>
                            'Approved');

```

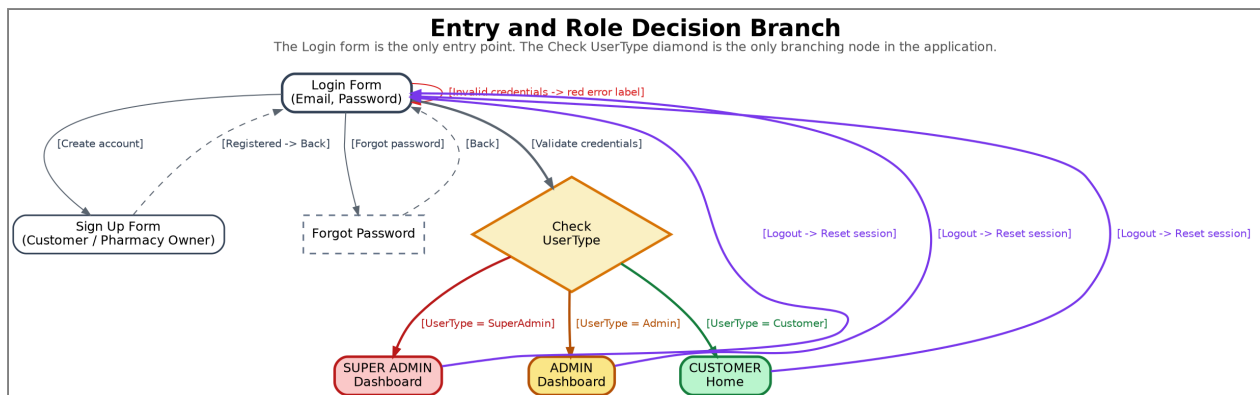
The password change verifies the current hash inside the same UPDATE, so a wrong current password simply updates no rows and the form reports failure without ever having read the stored hash into memory. The prescription statements are the Admin side of the same story: an order that still has a Pending prescription cannot be moved to Confirmed, which the last statement enforces with a NOT EXISTS clause rather than trusting the form.

8. Transition Table

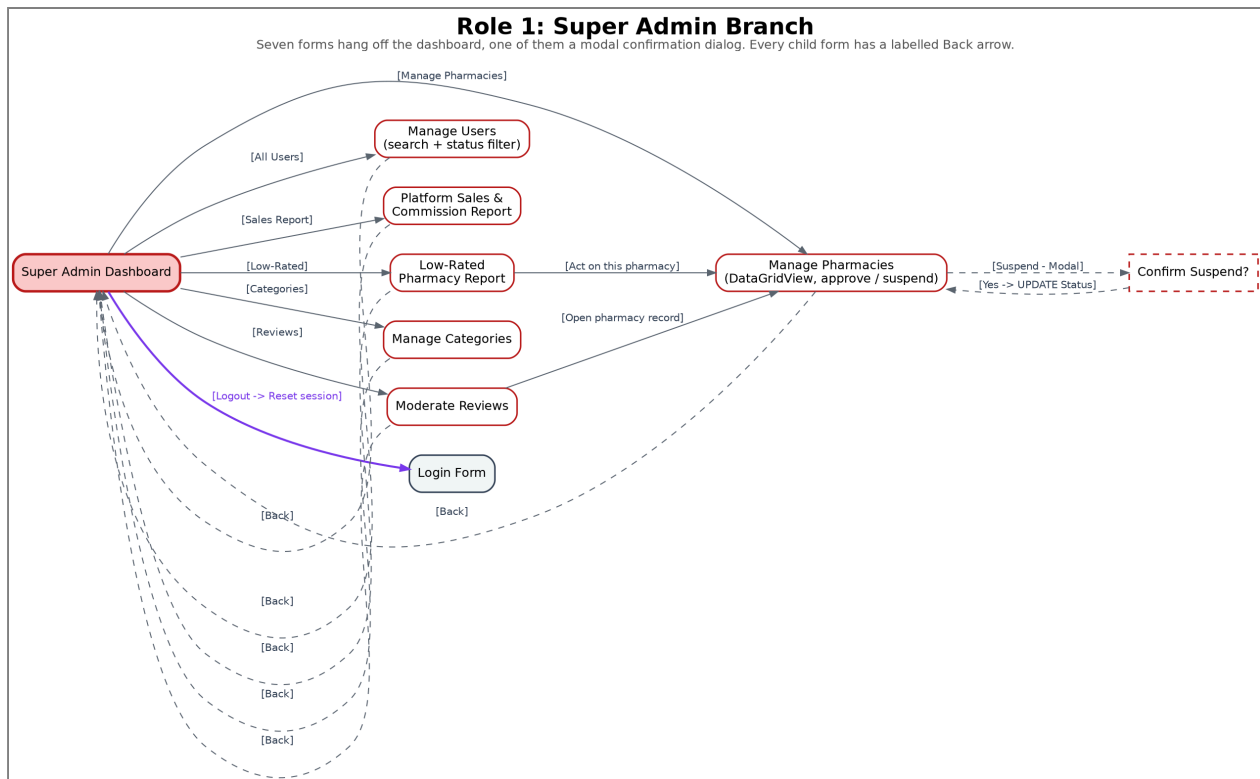
Every form in the application and every transition between them. The complete navigation diagram is given first, then the four regions at full size, then the screen flows themselves.



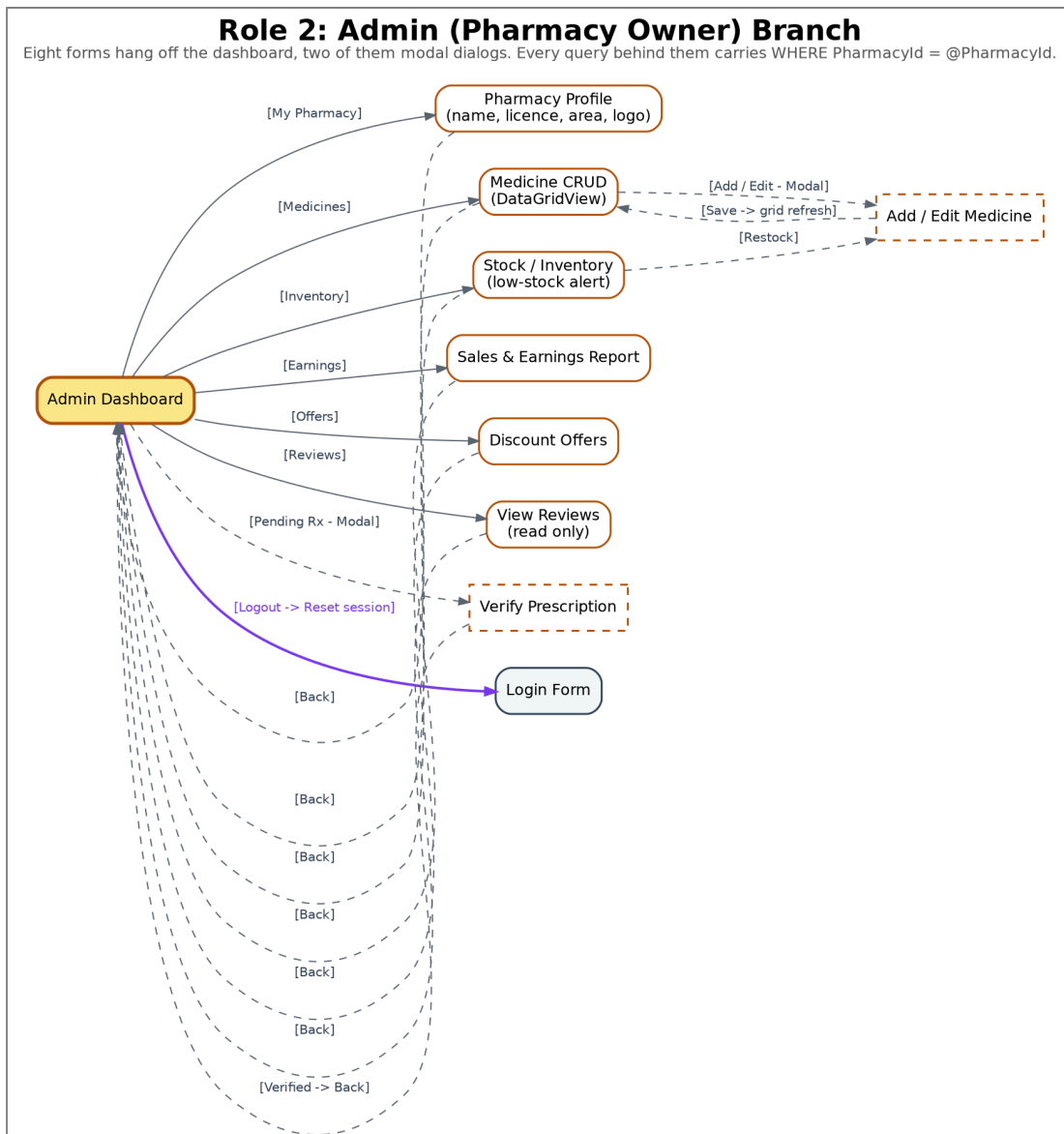
The complete navigation diagram. Solid borders are ordinary forms, dashed borders are modal dialogs, and every arrow carries the event that causes the transition.



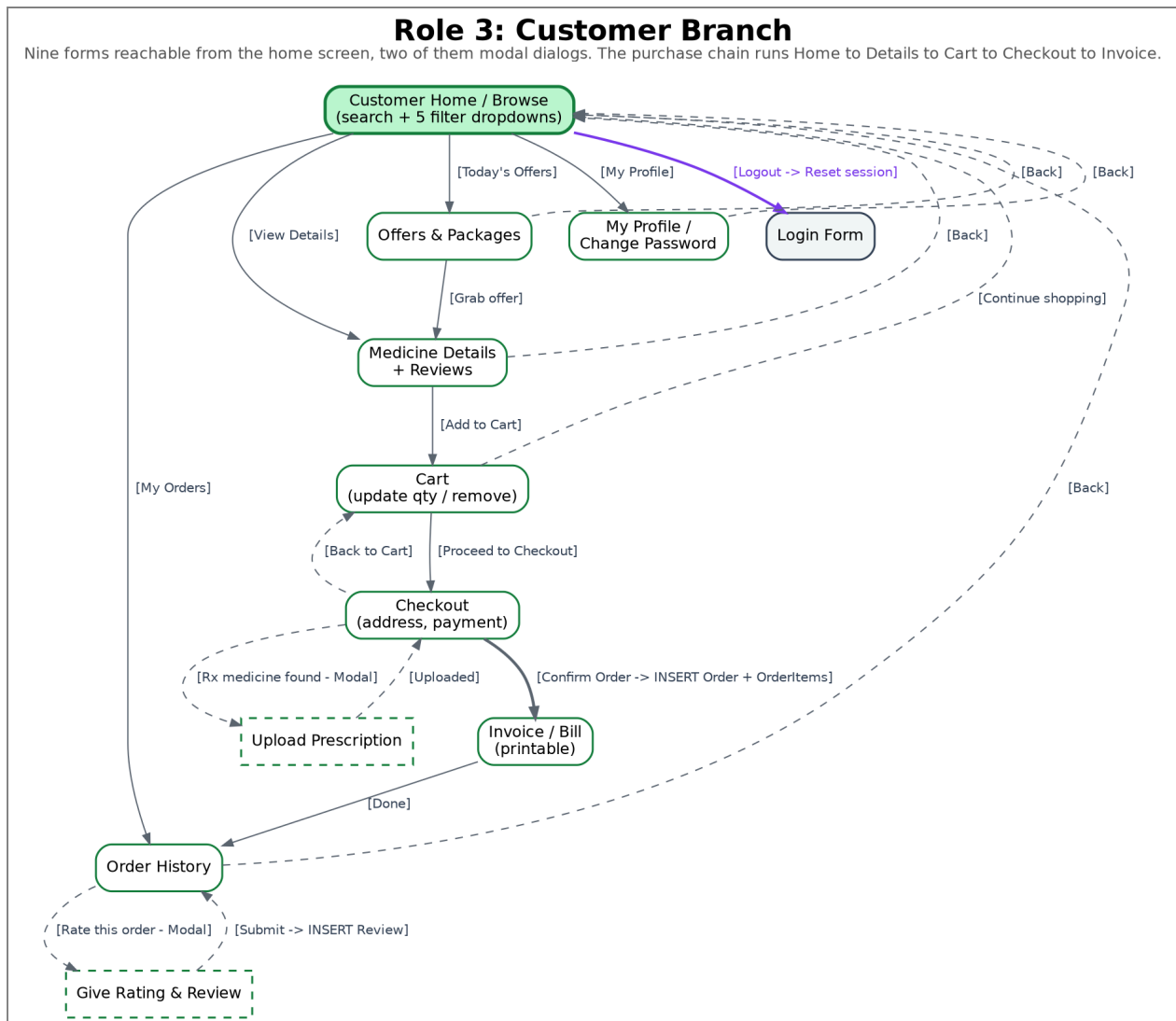
Entry and role decision. The Login form is the only entry point, and the Check UserType diamond is the only branching node in the application.



Super Admin branch. Seven forms hang off the dashboard, one of them a modal confirmation dialog, and each has a labelled Back arrow.



Admin branch. Eight forms hang off the dashboard, two of them modal dialogs. Every query behind them carries WHERE PharmacyId = @PharmacyId.



Customer branch. Nine forms are reachable from the home screen, two of them modal dialogs, and the purchase chain runs down the middle.

8.1 Login, sign up and role routing

- Shared entry, used by all three roles:

Login Form – The single entry point for all three roles. The screenshot shows the validation state: the password box is outlined in red, the error label reads 'Password must be at least 6 characters' and the Login button is disabled until the rule is satisfied.



Sign Up / Registration Form – Registration for both customers and pharmacy owners, chosen through the Register as dropdown. Two validation states are visible at once: an invalid email format and a password confirmation mismatch. A pharmacy owner registering here is stored with Status 'Pending', together with a Pending Pharmacies row, and neither becomes usable until the Super Admin approves it.

After the credentials are validated the login query returns the user's UserType, and a single decision opens one of the three dashboards below. No arrow ever crosses from one role branch into another.

8.2 Super Admin

- Super Admin dashboard and the forms reachable from it:

PharmaLink - Super Admin Dashboard [Atik Bin Masud]

File Platform Reports Moderation Help

SUPER ADMIN
Atik Bin Masud
PharmaLink Platform

Dashboard

- Manage Pharmacies
- Manage Users
- Platform Sales Report
- Low-Rated Pharmacies
- Manage Categories
- Moderate Reviews

Logout

Platform Overview Home > Dashboard

APPROVED PHARMACIES
3
1 pending approval

REGISTERED USERS
9
4 customers, 4 owners

TOTAL ORDERS
6
5 delivered, 1 placed

COMMISSION EARNED
₹ 215.98
from 5 delivered orders

Pharmacies Waiting for Approval

#	Pharmacy	Owner	Licence No	Area	Applied	Status	Action
4	New Life Pharmacy	Farhana Yeasmin	DGDA-CT-20015	Agrabad, Ctg	19 Aug 2026	Pending	Approve Reject

Pharmacies Rated Below 2.5 (needs attention)

Pharmacy	Owner	Area	Reviews	Avg Rating	Action
Lazz Care Pharmacy	Kazi Nazmul Haque	Dhanmondi	2	1.50 ★	View Suspend

Query: JOIN Pharmacies + Medicines + Reviews, GROUP BY pharmacy, HAVING AVG(Rating) < 2.5

Signed in as SuperAdmin Database: PharmaLinkDB Last refresh 10:42 AM

Super Admin Dashboard – The platform hub. Four summary tiles, a grid of pharmacies waiting for approval, and the low rated pharmacy panel that comes straight from the `HAVING AVG(Rating) < 2.5` query. The left menu is the entry point to every other Super Admin form.



PharmaLink - Manage Pharmacies

File Platform Reports Moderation Help

SUPER ADMIN
Atik Bin Masud
PharmaLink Platform

Manage Pharmacies

- Manage Users
- Platform Sales Report
- Low-Rated Pharmacies
- Manage Categories
- Moderate Reviews

Logout

Manage Pharmacy Owners Home > Manage Pharmacies

Search & Filter

Search Status Area

ID	Pharmacy Name	Owner	Licence No	Area	Contact	Commission	Avg Rating	Status
1	Mitford Pharma	Mohammad Rafiqul	DGDA-DH-10021	Mitford	02955000021	8.00 %	4.67 ★	Approved
2	Shahbagh Medicine Hub	Sultana Razia	DGDA-DH-10044	Shahbagh	02955000044	8.00 %	2.00 ★	Approved
3	Lazz Care Pharmacy	Kazi Nazmul Haque	DGDA-DH-10077	Dhanmondi	02955000077	10.00 %	1.50 ★	Approved
4	New Life Pharmacy	Farhana Yeasmin	DGDA-CT-20015	Agrabad	03155000015	8.00 %	–	Pending

Row 3 selected (Lazz Care Pharmacy, average rating 1.50). Suspending sets Pharmacies.Status = 'Suspended', Users.Status = 'Suspended' and hides every medicine of that pharmacy from customers. Historical orders are kept.

4 rows loaded Row 3 selected SELECT ... FROM Pharmacies JOIN Users

Super Admin: Manage Pharmacies – A DataGridView of every pharmacy with search and two ComboBox filters. Approve, Suspend and Delete act on the selected row only. The note under the grid explains that suspension updates three tables and deletes nothing.



PharmaLink - Platform Sales & Commission Report

File Platform Reports Moderation Help

SUPER ADMIN
Atik Bin Masud
PharmaLink Platform

Dashboard
Manage Pharmacies
Manage Users
Platform Sales Report
Low-Rated Pharmacies
Manage Categories
Moderate Reviews

Logout

Platform Sales & Commission Home > Reports > Sales

Report Filter
Date From 01 Aug 2026 To 23 Aug 2026 Area All Areas Status Delivered Generate Export CSV

GROSS SALES
b 2,598.50
5 delivered orders

COMMISSION
b 215.98
8-10 % per pharmacy

UNITS SOLD
95
9 line items

AVG ORDER VALUE
b 519.70
across all pharmacies

Pharmacy	Area	Orders	Units Sold	Gross Sales (v)	Commission (v)	Avg Item Price
Shahbagh Medicine Hub	Shahbagh	1	26	1,910.00	152.80	618.67
Lazz Care Pharmacy	Dhanmondi	2	4	405.00	40.50	107.50
Mitford Pharma	Mitford	2	65	283.50	22.68	11.47
TOTAL		5	95	2,598.50	215.98	-

SELECT ... FROM Pharmacies JOIN Orders JOIN OrderItems GROUP BY PharmacyId ORDER BY SUM(Subtotal) DESC

Report generated for 01 Aug - 23 Aug 2026 3 pharmacies

Super Admin: Platform Sales and Commission Report – Date range, area and status filters feed the JOIN plus GROUP BY earnings query. Tiles show gross sales, commission, units sold and average order value; the grid breaks the same figures down per pharmacy with a bold total row.



PharmaLink - Moderate Reviews

File Platform Reports Moderation Help

SUPER ADMIN
Atik Bin Masud
PharmaLink Platform

Dashboard
Manage Pharmacies
Manage Users
Platform Sales Report
Low-Rated Pharmacies
Manage Categories
Moderate Reviews

Logout

Moderate Reviews Home > Moderation > Reviews

Filter
Rating 1 and 2 star only Pharmacy All Pharmacies Visibility Visible Apply Reset

ID	Reviewer	Medicine	Pharmacy	Rating	Comment	Date	Verified Order
5	Rahim Uddin	Pediamin 100ml	Lazz Care Pharmacy	★ 1	Syrup arrived leaking and the seal was broken.	13 Aug 2026	#1004
4	Nusrat Jahan	Accu-Chek Active	Shahbagh Medicine Hub	★★ 2	Glucometer works but the box was already opened.	10 Aug 2026	#1003
6	Shakib Al Hasan	Cef-3 1g	Lazz Care Pharmacy	★★ 2	Delivery took two days for an injection, too slow.	17 Aug 2026	#1005

Hide Review Warn Pharmacy Open Pharmacy Record Back

Reviews are hidden (IsHidden = 1) rather than deleted, so the audit trail and the pharmacy's rating history stay complete. Pharmacy owners can read reviews but can never remove them.

3 low-rating reviews Row 5 selected Moderation queue

Super Admin: Moderate Reviews – The moderation queue, filtered by default to one and two star reviews. Each row carries the order number that verifies the purchase, so a review always traces back to a real delivered order. Hide Review sets IsHidden to 1 rather than deleting, so the pharmacy's rating history stays complete and the decision can be reversed.

8.3 Admin (Pharmacy Owner)

- Medicine management, with the Add dialog opening over the grid:

PharmaLink - Medicines [Mitford Pharma]

File Inventory Reports Offers Help

PHARMACY OWNER
Mohammad Rafiqul
Mitford Pharma, Dhaka

Dashboard
My Pharmacy Profile
Medicines
Stock / Inventory
Sales & Earnings
Discount Offers
Customer Reviews
Prescriptions

Logout

My Medicines

Mitford Pharma > Medicines

Search & Filter

Search Category Stock

ID	Medicine	Generic	Category	Manufacturer	Strength	Price (₳)	Stock	Min	Rx	Expiry
1	Napa	Paracetamol	Painkiller	Beximco	500mg	1.20	850	100	No	30 Jun 2027
2	Azithro	Azithromycin	Antibiotic	Square	500mg	35.00	60	40	Yes	31 Jan 2027
3	Cavit-D	Calcium + Vit D3	Vitamin	Renata	500mg	8.50	300	50	No	31 Mar 2028

Every query on this form carries WHERE PharmacyId = 1. Shahbagh Medicine Hub's rows can never appear here. That is how data isolation is enforced.

3 of 3 medicines shown PharmacyId = 1 (Mitford Pharma) Row 1 selected

Admin: Medicine CRUD – The pharmacy owner's inventory grid with Add, Update, Delete and Create Offer. The hint under the buttons states the isolation rule that every query on this form carries WHERE PharmacyId = 1, so another pharmacy's rows can never appear.



PharmaLink - Medicines [Mitford Pharma]

File Inventory Reports Offers Help

PHARMACY OWNER
Mohammad Rafiqul
Mitford Pharma, Dhaka

Dashboard
My Pharmacy Profile
Medicines
Stock / Inventory
Sales & Earnings
Discount Offers
Customer Reviews
Prescriptions

Logout

My Medicines

Mitford Pharma > Medicines > Add

Medicine Details

Medicine Name *

Generic Name *

Category *

Manufacturer *

Strength ☒ Requires prescription

Pricing & Stock

Unit Price (₳) * Stock * Min Stock

Expiry Date *

1 validation error Save is disabled until all fields are valid

Modal dialog open Add Medicine

Admin: Add Medicine (modal dialog with validation) – The modal drawn with a dashed border in the navigation diagram. Unit Price is negative, so the field is outlined in red, the error names the CHECK constraint that would reject the value at the database as well, and Save is greyed out.

• Inventory and earnings:

PharmaLink - Stock / Inventory Dashboard [Lazz Care Pharmacy]

FileInventoryReportsOffersHelp

PHARMACY OWNER

Kazi Nazmul Haque

Lazz Care Pharmacy, Dhaka

Dashboard

My Pharmacy Profile

Medicines

Stock / Inventory

Sales & Earnings

Discount Offers

Customer Reviews

Prescriptions

Logout

Stock & Inventory

Lazz Care Pharmacy > Inventory

MEDICINES LISTED

4

all active

LOW STOCK ALERTS

2

below minimum level

UNITS SOLD

4

this month

STOCK VALUE

৳ 17,926

at current price

▲ Low Stock Alert (Stock < MinStock)

Medicine	Generic	Category	In Stock	Min Level	Shortfall	Expiry	Action
Zinc-B	Zinc Sulphate	Vitamin	22	45	23	31 May 2027	Restock
Cef-3	Ceftriaxone	Antibiotic	18	25	7	31 Dec 2026	Restock

All Medicines

Medicine	Stock	Min	Sold	Status
Pediamin 100ml	140	30	3	Healthy
Tufnil 200mg	200	40	0	Healthy
Zinc-B 20mg	22	45	0	Low Stock
Cef-3 1g	18	25	1	Low Stock

SELECT ... FROM Medicines JOIN Categories WHERE PharmacyId = @PharmacyId AND Stock < MinStock

2 low stock alerts PharmacyId = 3 (Lazz Care Pharmacy)

Auto refresh every 60 s

Admin: Stock and Inventory Dashboard – The low stock alert panel lists medicines where Stock is below MinStock, tinted red, with the shortfall in units. Below it, the full inventory grid marks each medicine as Healthy or Low Stock.



PharmaLink - Sales & Earnings Report [Mitford Pharma]

FileInventoryReportsOffersHelp

PHARMACY OWNER

Mohammad Rafiqul

Mitford Pharma, Dhaka

Dashboard

My Pharmacy Profile

Medicines

Stock / Inventory

Sales & Earnings

Discount Offers

Customer Reviews

Prescriptions

Logout

Sales & Earnings

Mitford Pharma > Reports

Filter

From

01 Aug 2026

To

23 Aug 2026

Medicine

All Medicines

Generate

Print

GROSS SALES

৳ 283.50

2 orders

PLATFORM COMMISSION

৳ 22.68

8 % of gross

NET EARNINGS

৳ 260.82

payable to pharmacy

UNITS SOLD

65

4 line items

Order #	Date & Time	Customer	Medicine	Qty	Unit Price	Subtotal	Payment	Status
1001	02 Aug 2026, 10:15 AM	Rahim Uddin	Napa 500mg	30	1.20	36.00	bKash	Delivered
1001	02 Aug 2026, 10:15 AM	Rahim Uddin	Azithro 500mg	5	35.00	175.00	bKash	Delivered
1002	05 Aug 2026, 06:40 PM	Karim Sheikh	Napa 500mg	25	1.20	30.00	Cash on Delivery	Delivered
1002	05 Aug 2026, 06:40 PM	Karim Sheikh	Cavit-D 500mg	5	8.50	42.50	Cash on Delivery	Delivered
TOTAL				65	-	283.50		

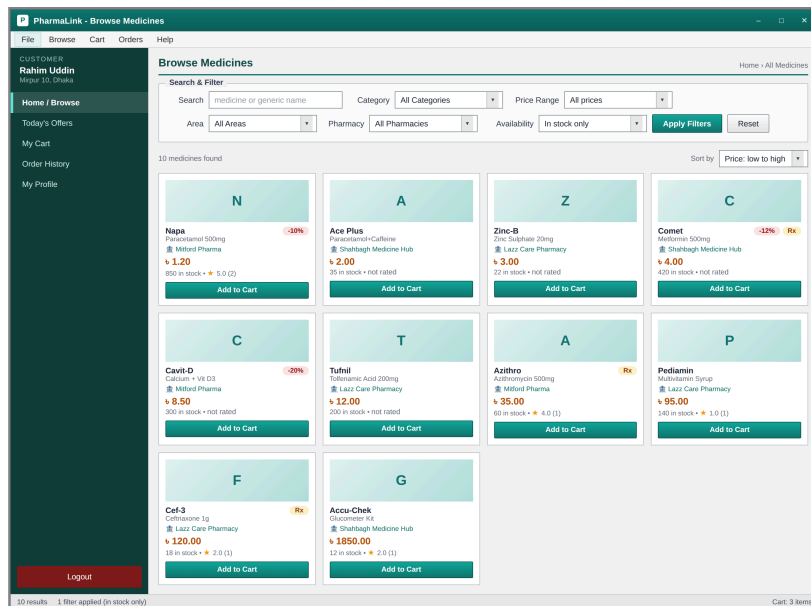
4 line items 01 Aug - 23 Aug 2026

Commission rate 8.00 %

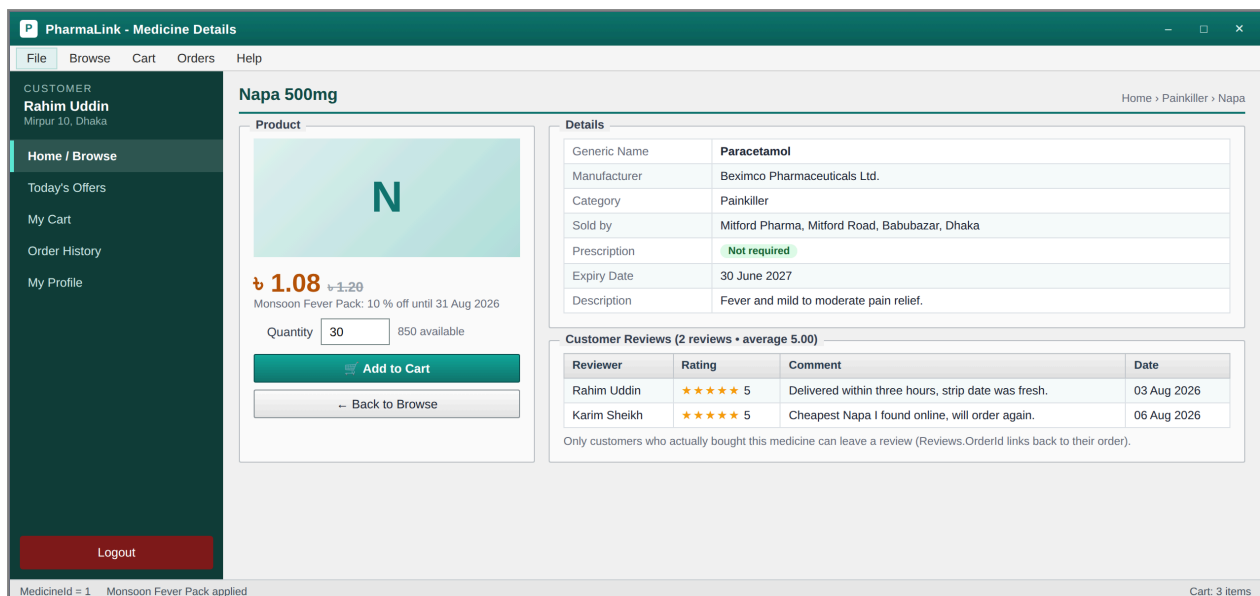
Admin: Sales and Earnings Report – Who bought what, when, at what unit price. Tiles separate gross sales, platform commission and net earnings so the owner can see exactly what PharmaLink deducts. The commission figure is read from the frozen Orders.CommissionAmount column.

8.4 Customer

- The purchase chain, from browsing to the invoice:



Customer: Home and Browse – The catalogue across every approved pharmacy, with a keyword search box and five ComboBox filters: category, price range, area, pharmacy and availability. Prescription only items are badged Rx and discounted items carry a percentage badge.



Customer: Medicine Details and Reviews – Manufacturer, strength, expiry date, selling pharmacy and prescription requirement, with the discounted price shown beside the struck through original. Underneath, the reviews grid comes from the join of Reviews and Users.



PharmaLink - My Cart

File Browse Cart Orders Help

CUSTOMER

Rahim Uddin

Mirpur 10, Dhaka

Home / Browse

Today's Offers

My Cart

Order History

My Profile

Logout

My Cart

Home > Cart

Medicine	Generic	Pharmacy	List Price (৳)	Offer	You Pay (৳)	Quantity	Subtotal (৳)	Remove
Cavit-D 500mg	Calcium + Vit D3	Mitford Pharma	8.50	20 %	6.80	- 20 +	136.00	×
Napa 500mg	Paracetamol	Mitford Pharma	1.20	10 %	1.08	- 30 +	32.40	×
Pediamin 100ml	Multivitamin Syrup	Lazz Care Pharmacy	95.00	–	95.00	- 1 +	95.00	×

Note

Your cart contains medicines from 2 different pharmacies, so checkout runs once per pharmacy and creates 2 orders: Mitford Pharma (৳ 168.40 + 60 delivery) and Lazz Care Pharmacy (৳ 95.00 + 60 delivery). Delivery is charged once per pharmacy. Prices shown in the You Pay column already include today's offers. No item in this cart needs a prescription.

Order Summary

Items at list price (3) ৳ 301.00

Offer discounts - ৳ 37.60

Items total ৳ 263.40

Delivery (2 pharmacies × 60) ৳ 120.00

Payable ৳ 383.40

Proceed to Checkout --

-- Continue Shopping

3 items in cart CustomerId = 6

Total payable Tk 383.40

Customer: Cart – One row per medicine with a quantity spinner and a subtotal, plus a summary panel showing items total, discount, delivery charge and the payable amount. The note explains that a cart spanning two pharmacies produces one order per pharmacy.



PharmaLink - Checkout

File Browse Cart Orders Help

CUSTOMER

Rahim Uddin

Mirpur 10, Dhaka

Home / Browse

Today's Offers

My Cart

Order History

My Profile

Logout

Checkout – Order 1 of 2 (Mitford Pharma)

Home > Cart > Checkout

Delivery Details

Full Name

Rahim Uddin

Mobile

01811000006

Delivery Address *

House 7, Road 3, Mirpur 10, Dhaka

Area

Mirpur

Same day delivery available

Payment Method

☒ bKash
 ☐ Nagad
 ☐ Card
 ☐ Cash on Delivery

bKash Number *

0181100

Enter a valid 11 digit bKash number.

Order Review

Item	Qty	৳
Cavit-D 500mg (~20 %)	20	136.00
Napa 500mg (~10 %)	30	32.40

The Pediamin line is sold by Lazz Care Pharmacy and becomes a second order after this one is confirmed.

List price ৳ 206.00

Offer discounts - ৳ 37.60

Items total ৳ 168.40

Delivery ৳ 60.00

Payable ৳ 228.40

✓ Confirm Order

-- Back to Cart

On confirm, for this pharmacy only: INSERT Orders --

INSERT OrderItems -- UPDATE Medicines stock --

DELETE the Mitford lines from Cart, all inside one transaction. Checkout then reopens for Lazz Care Pharmacy.

1 validation error Confirm Order disabled

CustomerId = 6

Customer: Checkout – The Mitford Pharma half of the basket, headed Order 1 of 2. Delivery details and payment method on the left, the order review on the right with the offer discounts already applied. The bKash number is incomplete, so it is outlined in red and Confirm Order is disabled. The hint lists the five statements the Confirm button runs inside one transaction, all of them scoped to this pharmacy.



PharmaLink - Invoice / Bill

FileBrowseCartOrdersHelp

CUSTOMER

Rahim Uddin

Mirpur 10, Dhaka

Home / Browse

Today's Offers

My Cart

Order History

My Profile

Logout

Invoice

Home > Orders > Invoice #1006

PharmaLink

Online Pharmacy Marketplace • Banani, Dhaka 1213

support@pharmalink.com.bd • 09600-PHARMA

INVOICE #1006

Date: 23 August 2026, 08:12 PM

Payment: bKash (TxnID 8FQ4KLM21)

Paid

Billed To

Rahim Uddin

House 7, Road 3, Mirpur 10, Dhaka

01811000006

Sold By

Mitford Pharma

22 Mitford Road, Babubazar, Dhaka 1100

Licence DGDA-DH-10021

#	Medicine	Generic	Qty	Unit Price (৳)	Subtotal (৳)
1	Cavit-D 500mg	Calcium + Vit D3	20	6.80	136.00
2	Napa 500mg	Paracetamol	30	1.08	32.40

List price: ৳ 206.00 | Offer discounts: - ৳ 37.60 | Items total: ৳ 168.40 | Delivery: ৳ 60.00

Grand Total: ৳ 228.40

Platform commission (8 % of the ৳ 168.40 items total) ৳ 13.47 is deducted from the pharmacy, not from the customer. No commission is charged on delivery. Medicines once sold are not returnable except for damaged or wrongly dispatched items. This is a computer generated invoice.

Print

Save as PDF

Back to Order History

Order #1006 of 2 placed successfully

Estimated delivery 24 Aug 2026

Customer: Invoice / Bill – The printable bill generated after checkout, carrying the order number, both addresses, the pharmacy licence number, the line items and the grand total. The footer notes that the platform commission is deducted from the pharmacy, not the customer.



PharmaLink - Order History

FileBrowseCartOrdersHelp

CUSTOMER

Rahim Uddin

Mirpur 10, Dhaka

Home / Browse

Today's Offers

My Cart

Order History

My Profile

Logout

My Orders

Home > Order History

Filter

Status

All

▼

Pharmacy

All Pharmacies

▼

Date Range

Last 30 days

▼

Apply

Reset

Order #	Date	Pharmacy	Items	Total (৳)	Payment	Status	Invoice	Review
1007	23 Aug 2026	Lazz Care Pharmacy	1	155.00	bKash	Placed	View	after delivery
1006	23 Aug 2026	Mitford Pharma	2	228.40	bKash	Placed	View	after delivery
1004	12 Aug 2026	Lazz Care Pharmacy	1	345.00	Nagad	Delivered	View	reviewed
1001	02 Aug 2026	Mitford Pharma	2	271.00	bKash	Delivered	View	reviewed

View Invoice

★ Rate & Review

Reorder

Back

Orders 1006 and 1007 are the two halves of one 23 August basket, split by pharmacy at checkout, each with its own delivery charge and its own invoice. The Rate button is enabled only for orders whose Status is 'Delivered' and that have not been reviewed yet, which the UNIQUE constraint on (CustomerId, MedicineId, OrderId) also enforces.

4 orders CustomerId = 6

Total spent Tk 999.40

Customer: Order History – Past orders with status pills, an invoice link and a Rate and Review action. Orders 1006 and 1007 are the two halves of one 23 August basket, split by pharmacy at checkout. The hint explains that the Rate button is enabled only for delivered orders that have not already been reviewed, which the UNIQUE constraint on Reviews also enforces.

• Offers and profile:

PharmaLink - Offers & Packages

File Browse Cart Orders Help

CUSTOMER
Rahim Uddin
Mirpur 10, Dhaka

Home / Browse

Today's Offers

My Cart

Order History

My Profile

Logout

Today's Offers Home > Offers

Filter

Category All Categories Pharmacy All Pharmacies Minimum 10 % or more Apply Reset

Offer	Medicine	Pharmacy	Original (v)	Discount	You Pay (v)	Valid Until	Action
Bone Health Week	Cavit-D 500mg	Mitford Pharma	8.50	20 %	6.80	25 Aug 2026	Add to Cart
Diabetes Awareness	Comet 500mg	Shahbagh Medicine Hub	4.00	12 %	3.52	15 Sep 2026	Add to Cart
Monsoon Fever Pack	Napa 500mg	Mitford Pharma	1.20	10 %	1.08	31 Aug 2026	Add to Cart

Only offers where today falls between StartDate and EndDate, the medicine is in stock, and the pharmacy is Approved are shown. Expired offers such as the July glucometer offer are filtered out automatically.

3 active offers today 23 Aug 2026 1 expired offer hidden

Customer: Offers and Packages – The three offers running on 23 August 2026, with the discounted price already calculated by the query so the same number appears here, on the details screen and in the cart. Offers whose EndDate has passed, such as the July glucometer offer in the sample data, are filtered out at the database rather than in the form.



PharmaLink - My Profile

File Browse Cart Orders Help

CUSTOMER
Rahim Uddin
Mirpur 10, Dhaka

Home / Browse

Today's Offers

My Cart

Order History

My Profile

Logout

My Profile Home > Profile

Personal Information

Full Name Rahim Uddin

Email rahim@gmail.com cannot be changed

Mobile 01811000006

Address House 7, Road 3, Mirpur 10, Dhaka

Member Since 14 July 2026

Save Changes Cancel

Change Password

Current *****

New Password *****

Confirm New *****

Minimum 6 characters, at least one digit.

Update Password

The new password is salted and hashed with SHA-256 before it is written to Users.PasswordHash. Plain text is never stored.

Profile loaded Userid = 6 No unsaved changes

Customer: My Profile and Change Password – Profile editing on the left and password change on the right. Email is read only because it is the login identifier. The note records that the new password is salted and hashed with SHA-256 before it reaches the database.

9. Features and Facilities

- One login for three roles:** Every user signs in through the same form. The login query returns the UserType, and that single value decides whether the Super Admin dashboard, the pharmacy owner dashboard or the customer home screen opens. There is no separate administrator entry point to protect.

2. **Licence checked pharmacy onboarding:** A pharmacy owner who registers stays in Pending status, and none of his medicines are visible to customers, until the Super Admin has checked the DGDA drug licence number and approved him.
3. **Data isolation between pharmacies:** Every query on an Admin form carries WHERE PharmacyId = @PharmacyId, taken from the session at login. One pharmacy owner can never read another owner's medicines, orders or earnings, and the rule is enforced in the query rather than by hiding controls in the interface.
4. **Search and five way filtering:** Customers search by brand name, generic name or manufacturer, so a doctor's chit that says paracetamol finds Napa. Results are narrowed by category, price range, area, pharmacy and availability, all through ComboBox controls rather than free text.
5. **Low stock alerts:** Each medicine carries its own minimum stock level, so the alert compares two columns of the same row instead of using a fixed threshold. Ten boxes of a glucometer is plenty; ten strips of a painkiller is nothing. The alert shows the shortfall in units so the owner knows how much to order.
6. **Prescription control:** A medicine flagged as prescription only cannot be checked out until the customer uploads a photograph of the doctor's prescription, and the order cannot be confirmed until the pharmacy has verified that image.
7. **Time limited discount offers:** A pharmacy owner sets a percentage discount with a start and an end date. The discounted price is calculated inside the query, so the same figure appears on the offers screen, the details screen, the cart and the invoice with no chance of the four disagreeing.
8. **Transactional checkout:** Writing the order, writing its line items, reducing stock and clearing the cart all happen inside one transaction, scoped to one pharmacy. A basket spanning two pharmacies becomes two orders, each with its own delivery charge and its own invoice.
9. **Verified reviews and moderation:** A review points back at the order it came from, so only a customer who actually received the medicine can rate it, and only once per purchase. The Super Admin can hide an abusive review but never delete it, so the rating history stays complete.
10. **Commission frozen at the point of sale:** The commission is calculated once at checkout from the pharmacy's own rate and stored on the order row. A price change next month cannot rewrite what was owed on last month's sales, and the earnings report never has to be reconciled by hand.
11. **Printable invoices and order history:** Every completed order produces a printable bill carrying both addresses, the pharmacy licence number, the line items at the price charged, and the grand total. Customers can reopen any past invoice from their order history.
12. **Passwords never stored in plain text:** Passwords are salted and hashed with SHA-256 before they reach the database, and a password change verifies the current hash inside the same UPDATE, so a wrong entry simply updates no rows.

10. Conclusion

PharmaLink began as an ordinary observation about buying medicine late at night in Dhaka and ended as a complete blueprint for a three tier marketplace. The design work in this report covers a case study grounded in a real domain, a full set of functional requirements and user stories for all three roles, a navigation diagram in which every form is reachable and every form has a way back, a ten table database normalised to third normal form with a mandatory junction table for order line items, a runnable SQL script whose queries sit behind named features rather than existing for their own sake, and eighteen consistent form designs including validation states.

The three artefacts agree with each other, and that was the hardest part of the work. Every form in the navigation diagram that reads or writes data has at least one statement behind it in the SQL script, the traceability table in the requirements section maps each of the thirty functional requirements to its form and its query, and every query touches only tables and columns that exist in the schema. Where the report claims that a pharmacy owner cannot see another pharmacy's data, the claim is backed by a specific clause, `WHERE PharmacyId = @PharmacyId`, that appears in the queries themselves. Where it claims a customer cannot review the same purchase twice, the claim is backed by a `UNIQUE` constraint on three columns.

What the group takes away from this phase is that the design decides how hard the coding will be. Choosing a junction table now means the checkout code will be simple later. Storing the commission on the order row now means the earnings report will never have to be reconciled by hand. Keeping the category list in its own table now means a rename will be one `UPDATE` instead of a hundred. The coding phase will show whether those judgements were right, and the plan under Future Work is written so that the group can start on it immediately.

10.1 Future Work

This submission is the design phase. The coding phase will implement the blueprint in the order below, and the challenges we already expect are noted against each item.

1. **Login and role routing first.** The single decision node in the navigation diagram becomes a switch on `UserType` inside `LoginForm`. The challenge is session state: we need one static `Session` class holding the user id, user type and `PharmacyId` so that every later query can filter by it, and it has to be cleared properly on logout.
2. **Repository layer before any form.** Each table gets a repository class with parameterised queries. Getting this right early is what stops SQL injection and stops query strings being scattered through button click handlers.
3. **Admin data isolation.** Every Admin side query must carry `WHERE PharmacyId = Session.PharmacyId`. The expected difficulty is remembering it consistently, so the plan is to pass the `PharmacyId` into the repository constructor rather than into each method, which makes it impossible to forget.
4. **Checkout as a single transaction.** Inserting the order, inserting the line items, decrementing stock and clearing the cart must all succeed or all fail. Concurrent orders on

the last unit of a medicine are the real risk here, and we plan to re-check stock inside the transaction before committing.

5. **DataGridView formatting.** Low stock rows in red and delivered orders in green need the `CellFormatting` event rather than manual painting, and refreshing the grid without losing the selected row takes care.
6. **Reporting queries.** The commission and low rated pharmacy reports use JOIN with GROUP BY and HAVING. Testing them against the seeded data before wiring them to a form will save time.

Beyond the course requirements, the features we would like to add are a delivery rider role with live order tracking, an SMS notification when an order status changes, a refill reminder for patients on long term medication, a bKash payment gateway integration instead of a manually entered transaction id, and a Bangla language toggle for the customer screens.